

Document converter

This is an automatic generated API reference of the main components of Docling.

mod document_converter

Classes:

- **DocumentConverter** – Convert documents of various input formats to Docling documents.
- **ConversionResult** –
- **ConversionStatus** –
- **FormatOption** –
- **InputFormat** – A document format supported by document backend parsers.
- **PdfFormatOption** –
- **ImageFormatOption** –
- **StandardPdfPipeline** – High-performance PDF pipeline with multi-threaded stages.
- **WordFormatOption** –
- **PowerpointFormatOption** –
- **MarkdownFormatOption** –
- **AsciiDocFormatOption** –
- **HTMLFormatOption** –
- **SimplePipeline** – SimpleModelPipeline.

class DocumentConverter

```
DocumentConverter(allowed_formats: Optional[list[InputFormat]] = None, format_options:
Optional[dict[InputFormat, FormatOption]] = None)
```

Convert documents of various input formats to Docling documents.

DocumentConverter is the main entry point for converting documents in Docling. It handles various input formats (PDF, DOCX, PPTX, images, HTML, Markdown, etc.) and provides both single-document and batch conversion capabilities.

The conversion methods return a **ConversionResult** instance for each document, which wraps a **DoclingDocument** object if the conversion was successful, along with metadata about the conversion process.

Parameters:

- **allowed_formats** (`Optional[list[InputFormat]]`, default: `None`) – List of allowed input formats. By default, any format supported by Docling is allowed.

- `format_options` (Optional[dict[`InputFormat`, `FormatOption`]] , default: `None`) – Dictionary of format-specific options.

Examples:

Create a converter with default settings (all formats allowed):

```
>>> converter = DocumentConverter()
```

Allow only PDF and DOCX formats:

```
>>> from docling.datamodel.base_models import InputFormat
>>> converter = DocumentConverter(
...     allowed_formats=[InputFormat.PDF, InputFormat.DOCX]
... )
```

Customize pipeline options for PDF:

```
>>> from docling.datamodel.pipeline_options import PdfPipelineOptions
>>> converter = DocumentConverter(
...     format_options={
...         InputFormat.PDF: PdfFormatOption(
...             pipeline_options=PdfPipelineOptions()
...         ),
...     },
... )
```

Methods:

- `convert` – Convert one document fetched from a file path, URL, or DocumentStream.
- `convert_all` – Convert multiple documents from file paths, URLs, or DocumentStreams.
- `convert_string` – Convert a document given as a string using the specified format.
- `initialize_pipeline` – Initialize the conversion pipeline for the selected format.

attr `allowed_formats` instance-attribute

```
allowed_formats: list[InputFormat]
```

attr `format_to_options` instance-attribute

```
format_to_options: dict[InputFormat, FormatOption]
```

attr `initialized_pipelines` instance-attribute

```
initialized_pipelines: dict[tuple[Type[BasePipeline], str], BasePipeline]
```

meth `convert`

```
convert(source: Union[Path, str, DocumentStream], headers: Optional[dict[str, str]] = None, raises_on_error:
bool = True, max_num_pages: int = maxsize, max_file_size: int = maxsize, page_range: PageRange =
DEFAULT_PAGE_RANGE) -> ConversionResult
```

Convert one document fetched from a file path, URL, or DocumentStream.

Note: If the document content is given as a string (Markdown or HTML content), use the `convert_string` method.

Parameters:

- `source` (`Union[Path, str, DocumentStream]`) – Source of input document given as file path, URL, or `DocumentStream`.
- `headers` (`Optional[dict[str, str]]`, default: `None`) – Optional headers given as a dictionary of string key-value pairs, in case of URL input source.
- `raises_on_error` (`bool`, default: `True`) – Whether to raise an error on the first conversion failure. If `False`, errors are captured in the `ConversionResult` objects.
- `max_num_pages` (`int`, default: `maxsize`) – Maximum number of pages accepted per document. Documents exceeding this number will not be converted.
- `max_file_size` (`int`, default: `maxsize`) – Maximum file size to convert.
- `page_range` (`PageRange`, default: `DEFAULT_PAGE_RANGE`) – Range of pages to convert.

Returns:

- `ConversionResult` – The conversion result, which contains a `DoclingDocument` in the `document` attribute, and metadata about the conversion process.

Raises:

- `ConversionError` – An error occurred during conversion.

Examples:

Convert a local PDF file:

```
>>> from pathlib import Path
>>> converter = DocumentConverter()
>>> result = converter.convert("path/to/document.pdf")
>>> print(result.document.export_to_markdown())
```

Convert a document from a URL:

```
>>> result = converter.convert("https://example.com/paper.pdf")
```

Convert from an in-memory stream:

```
>>> from io import BytesIO
>>> from docling.datamodel.base_models import DocumentStream
>>> buf = BytesIO(b"<html><body>Hello</body></html>")
>>> stream = DocumentStream(name="page.html", stream=buf)
>>> result = converter.convert(stream)
```

meth `convert_all`

```
convert_all(source: Iterable[Union[Path, str, DocumentStream]], headers: Optional[dict[str, str]] = None,
raises_on_error: bool = True, max_num_pages: int = maxsize, max_file_size: int = maxsize, page_range:
PageRange = DEFAULT_PAGE_RANGE) -> Iterator[ConversionResult]
```

Convert multiple documents from file paths, URLs, or `DocumentStreams`.

Parameters:

- **source** (Iterable[Union[Path, str, DocumentStream]]) – Source of input documents given as an iterable of file paths, URLs, or DocumentStreams.
- **headers** (Optional[dict[str, str]], default: None) – Optional headers given as a (single) dictionary of string key-value pairs, in case of URL input source.
- **raises_on_error** (bool, default: True) – Whether to raise an error on the first conversion failure.
- **max_num_pages** (int, default: maxsize) – Maximum number of pages accepted per document. Documents exceeding this number will not be converted.
- **max_file_size** (int, default: maxsize) – Maximum file size in bytes. Documents exceeding this limit will be skipped.
- **page_range** (PageRange, default: DEFAULT_PAGE_RANGE) – Range of pages to convert in each document.

Yields:

- [ConversionResult](#) – The conversion results, each containing a [DoclingDocument](#) in the `document` attribute and metadata about the conversion process.

Raises:

- [ConversionError](#) – An error occurred during conversion.

Examples:

Convert a batch of local files:

```
>>> from pathlib import Path
>>> converter = DocumentConverter()
>>> paths = list(Path("docs/").glob("*.pdf"))
>>> for result in converter.convert_all(paths):
>>>     print(result.document.export_to_markdown()[ :100 ])
```

Convert with a file size limit of 20 MB:

```
>>> results = converter.convert_all(
...     paths, max_file_size=20 * 1024 * 1024
... )
```

meth `convert_string`

```
convert_string(content: str, format: InputFormat name: Optional[str] = None) -> ConversionResult
```

Convert a document given as a string using the specified format.

Only Markdown (``InputFormat.MD``) and HTML (``InputFormat.HTML``) formats are supported. The content is wrapped in a ``DocumentStream`` and passed to the main conversion pipeline.

Args:

`content`: The document content as a string.
`format`: The format of the input content.
`name`: The filename to associate with the document. If not provided, a timestamp-based name is generated. The appropriate file extension (``md`` or ``html``) is appended if missing.

Returns:

The conversion result, which contains a ``DoclingDocument`` in the ``document``

attribute, and metadata about the conversion process.

Raises:

`ValueError`: If format is neither ``InputFormat.MD`` nor ``InputFormat.HTML``.
`ConversionError`: An error occurred during conversion.

Examples:

Convert a Markdown string:

```
>>> from docling.datamodel.base_models import InputFormat
>>> converter = DocumentConverter()
>>> result = converter.convert_string(
...     "# Title
```

```
Some text.", format=InputFormat.MD ... ) >>> print(result.document.export_to_markdown())
```

Convert an HTML string:

```
>>> result = converter.convert_string(
...     "<h1>Title</h1><p>Some text.</p>",
...     format=InputFormat.HTML,
...     name="my_page",
... )
```

meth initialize_pipeline

```
initialize_pipeline(format: InputFormat)
```

Initialize the conversion pipeline for the selected format.

Parameters:

- **format** (`InputFormat`) – The input format for which to initialize the pipeline.

Raises:

- `ConversionError` – If no pipeline could be initialized for the given format.
- `RuntimeError` – If `artifacts_path` is set in `docling.datamodel.settings.settings` when required by the pipeline, but points to a non-directory file.
- `FileNotFoundError` – If local model files are not found.

class `ConversionResult` `pydantic-model`

Bases: `ConversionAssets`



```
{
  "$defs": {
    "AssembledUnit": {
      "properties": {
        "elements": {
          "default": [],
          "items": {
            "anyOf": [
              {
                "$ref": "#/$defs/TextElement"
              },
              {
                "$ref": "#/$defs/Table"
              },
              {
                "$ref": "#/$defs/FigureElement"
              },
              {
                "$ref": "#/$defs/ContainerElement"
              }
            ]
          },
          "title": "Elements",
          "type": "array"
        },
        "body": {
          "default": [],
          "items": {
            "anyOf": [
              {
                "$ref": "#/$defs/TextElement"
              },
              {
                "$ref": "#/$defs/Table"
              },
              {
                "$ref": "#/$defs/FigureElement"
              },
              {
                "$ref": "#/$defs/ContainerElement"
              }
            ]
          },
          "title": "Body",
          "type": "array"
        },
        "headers": {
          "default": [],
          "items": {
            "anyOf": [
              {
                "$ref": "#/$defs/TextElement"
              },
              {
                "$ref": "#/$defs/Table"
              },
              {
                "$ref": "#/$defs/FigureElement"
              },
              {
                "$ref": "#/$defs/ContainerElement"
              }
            ]
          },
          "title": "Headers",
```

```

        "type": "array"
    },
    },
    "title": "AssembledUnit",
    "type": "object"
},
"BaseMeta": {
    "additionalProperties": true,
    "description": "Base class for metadata.",
    "properties": {
        "summary": {
            "anyOf": [
                {
                    "$ref": "#/$defs/SummaryMetaField"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        }
    },
    "title": "BaseMeta",
    "type": "object"
},
"BitmapResource": {
    "description": "Model representing a bitmap resource with positioning and URI information.",
    "properties": {
        "index": {
            "default": -1,
            "title": "Index",
            "type": "integer"
        },
        "rect": {
            "$ref": "#/$defs/BoundingBox"
        },
        "mode": {
            "$ref": "#/$defs/ImageRefMode",
            "default": "placeholder"
        },
        "image": {
            "anyOf": [
                {
                    "$ref": "#/$defs/ImageRef"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "uri": {
            "anyOf": [
                {
                    "format": "uri",
                    "minLength": 1,
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "deprecated": true,
            "title": "Uri"
        }
    },
    "required": [
        "rect"
    ]
}

```

```

    },
    "title": "BitmapResource",
    "type": "object"
  },
  "BoundingBox": {
    "description": "BoundingBox.",
    "properties": {
      "l": {
        "title": "L",
        "type": "number"
      },
      "t": {
        "title": "T",
        "type": "number"
      },
      "r": {
        "title": "R",
        "type": "number"
      },
      "b": {
        "title": "B",
        "type": "number"
      },
      "coord_origin": {
        "$ref": "#/$defs/CoordOrigin",
        "default": "TOPLEFT"
      }
    },
    "required": [
      "l",
      "t",
      "r",
      "b"
    ],
    "title": "BoundingBox",
    "type": "object"
  },
  "BoundingBox": {
    "description": "Model representing a rectangular boundary with four corner points.",
    "properties": {
      "r_x0": {
        "title": "R X0",
        "type": "number"
      },
      "r_y0": {
        "title": "R Y0",
        "type": "number"
      },
      "r_x1": {
        "title": "R X1",
        "type": "number"
      },
      "r_y1": {
        "title": "R Y1",
        "type": "number"
      },
      "r_x2": {
        "title": "R X2",
        "type": "number"
      },
      "r_y2": {
        "title": "R Y2",
        "type": "number"
      },
      "r_x3": {
        "title": "R X3",
        "type": "number"
      },
      "r_y3": {

```



```

        "title": "R Y3",
        "type": "number"
    },
    "coord_origin": {
        "$ref": "#/$defs/CoordOrigin",
        "default": "BOTTOMLEFT"
    }
},
"required": [
    "r_x0",
    "r_y0",
    "r_x1",
    "r_y1",
    "r_x2",
    "r_y2",
    "r_x3",
    "r_y3"
],
"title": "BoundingBox",
"type": "object"
},
"ChartBar": {
    "description": "Represents a bar in a bar chart.\n\nAttributes:\n    label (str): The label for the bar.\n    values (float): The value associated with the bar.",
    "properties": {
        "label": {
            "title": "Label",
            "type": "string"
        },
        "values": {
            "title": "Values",
            "type": "number"
        }
    },
    "required": [
        "label",
        "values"
    ],
    "title": "ChartBar",
    "type": "object"
},
"ChartLine": {
    "description": "Represents a line in a line chart.\n\nAttributes:\n    label (str): The label for the line.\n    values (list[tuple[float, float]]): A list of (x, y) coordinate pairs\n    representing the line's data points.",
    "properties": {
        "label": {
            "title": "Label",
            "type": "string"
        },
        "values": {
            "items": {
                "maxItems": 2,
                "minItems": 2,
                "prefixItems": [
                    {
                        "type": "number"
                    },
                    {
                        "type": "number"
                    }
                ]
            },
            "type": "array"
        },
        "title": "Values",
        "type": "array"
    },
    "required": [

```

```

        "label": {
            "values":
        },
        "title": "ChartLine",
        "type": "object"
    },
    "ChartPoint": {
        "description": "Represents a point in a scatter chart.\n\nAttributes:\n    value (Tuple[float, float]): A\n(x, y) coordinate pair representing a point in a\n    chart.",
        "properties": {
            "value": {
                "maxItems": 2,
                "minItems": 2,
                "prefixItems": [
                    {
                        "type": "number"
                    },
                    {
                        "type": "number"
                    }
                ],
                "title": "Value",
                "type": "array"
            }
        },
        "required": [
            "value"
        ],
        "title": "ChartPoint",
        "type": "object"
    },
    "ChartSlice": {
        "description": "Represents a slice in a pie chart.\n\nAttributes:\n    label (str): The label for the\nslice.\n    value (float): The value represented by the slice.",
        "properties": {
            "label": {
                "title": "Label",
                "type": "string"
            },
            "value": {
                "title": "Value",
                "type": "number"
            }
        },
        "required": [
            "label",
            "value"
        ],
        "title": "ChartSlice",
        "type": "object"
    },
    "ChartStackedBar": {
        "description": "Represents a stacked bar in a stacked bar chart.\n\nAttributes:\n    label (list[str]):\nThe labels for the stacked bars. Multiple values are stored\n    in cases where the chart is \"double\nstacked,\" meaning bars are stacked both\n    horizontally and vertically.\n    values (list[tuple[str,\nint]]): A list of values representing different segments\n    of the stacked bar along with their label.",
        "properties": {
            "label": {
                "items": {
                    "type": "string"
                },
                "title": "Label",
                "type": "array"
            },
            "values": {
                "items": {
                    "maxItems": 2,
                    "minItems": 2,
                    "prefixItems": [

```

```

        "type": "string"
    },
    {
        "type": "integer"
    }
],
    "type": "array"
},
    "title": "Values",
    "type": "array"
}
},
"required": [
    "label",
    "values"
],
"title": "ChartStackedBar",
"type": "object"
},
"Cluster": {
    "properties": {
        "id": {
            "title": "Id",
            "type": "integer"
        },
        "label": {
            "$ref": "#/$defs/DocItemLabel"
        },
        "bbox": {
            "$ref": "#/$defs/BoundingBox"
        },
        "confidence": {
            "default": 1.0,
            "title": "Confidence",
            "type": "number"
        },
        "cells": {
            "default": [],
            "items": {
                "$ref": "#/$defs/TextCell"
            },
            "title": "Cells",
            "type": "array"
        },
        "children": {
            "default": [],
            "items": {
                "$ref": "#/$defs/Cluster"
            },
            "title": "Children",
            "type": "array"
        }
    },
    "required": [
        "id",
        "label",
        "bbox"
    ],
    "title": "Cluster",
    "type": "object"
},
"CodeItem": {
    "additionalProperties": false,
    "description": "CodeItem.",
    "properties": {
        "self_ref": {
            "pattern": "^#(?:/(\\w-+)(?:/(\\d+))?)?$",
            "title": "Self Ref",

```

```

    "type": "string"
  },
  "parent": {
    "anyOf": [
      {
        "$ref": "#/$defs/RefItem"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "children": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Children",
    "type": "array"
  },
  "content_layer": {
    "$ref": "#/$defs/ContentLayer",
    "default": "body"
  },
  "meta": {
    "anyOf": [
      {
        "$ref": "#/$defs/FloatingMeta"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "label": {
    "const": "code",
    "default": "code",
    "title": "Label",
    "type": "string"
  },
  "prov": {
    "default": [],
    "items": {
      "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
  },
  "source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
    "items": {
      "discriminator": {
        "mapping": {
          "track": "#/$defs/TrackSource"
        }
      },
      "propertyName": "kind"
    },
    "oneOf": [
      {
        "$ref": "#/$defs/TrackSource"
      }
    ]
  },
  "title": "Source",
  "type": "array"
}

```

```
    },
    "comments": {
      "default": [],
      "items": {
        "$ref": "#/$defs/FineRef"
      },
      "title": "Comments",
      "type": "array"
    },
    "orig": {
      "title": "Orig",
      "type": "string"
    },
    "text": {
      "title": "Text",
      "type": "string"
    },
    "formatting": {
      "anyOf": [
        {
          "$ref": "#/$defs/Formatting"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "hyperlink": {
      "anyOf": [
        {
          "format": "uri",
          "minLength": 1,
          "type": "string"
        },
        {
          "format": "path",
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Hyperlink"
    },
    "captions": {
      "default": [],
      "items": {
        "$ref": "#/$defs/RefItem"
      },
      "title": "Captions",
      "type": "array"
    },
    "references": {
      "default": [],
      "items": {
        "$ref": "#/$defs/RefItem"
      },
      "title": "References",
      "type": "array"
    },
    "footnotes": {
      "default": [],
      "items": {
        "$ref": "#/$defs/RefItem"
      },
      "title": "Footnotes",
      "type": "array"
    }
  }
}
```

```

    },
    "image": {
      "anyOf": [
        {
          "$ref": "#/$defs/ImageRef"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "code_language": {
      "$ref": "#/$defs/CodeLanguageLabel",
      "default": "unknown"
    }
  },
  "required": [
    "self_ref",
    "orig",
    "text"
  ],
  "title": "CodeItem",
  "type": "object"
},
"CodeLanguageLabel": {
  "description": "CodeLanguageLabel.",
  "enum": [
    "Ada",
    "Awk",
    "Bash",
    "bc",
    "C",
    "C#",
    "C++",
    "CMake",
    "COBOL",
    "CSS",
    "Ceylon",
    "Clojure",
    "Crystal",
    "Cuda",
    "Cython",
    "D",
    "Dart",
    "dc",
    "Dockerfile",
    "Elixir",
    "Erlang",
    "FORTRAN",
    "Forth",
    "Go",
    "HTML",
    "Haskell",
    "Haxe",
    "Java",
    "JavaScript",
    "JSON",
    "Julia",
    "Kotlin",
    "Lisp",
    "Lua",
    "Matlab",
    "MoonScript",
    "Nim",
    "OCaml",
    "ObjectiveC",
    "Octave",
    "PHP",

```

```

        "Pascal",
        "Perl",
        "Prolog",
        "Python",
        "Racket",
        "Ruby",
        "Rust",
        "SML",
        "SQL",
        "Scala",
        "Scheme",
        "Swift",
        "TypeScript",
        "unknown",
        "VisualBasic",
        "XML",
        "YAML"
    ],
    "title": "CodeLanguageLabel",
    "type": "string"
},
"ColorRGBA": {
    "description": "Model representing an RGBA color value.",
    "properties": {
        "r": {
            "maximum": 255,
            "minimum": 0,
            "title": "R",
            "type": "integer"
        },
        "g": {
            "maximum": 255,
            "minimum": 0,
            "title": "G",
            "type": "integer"
        },
        "b": {
            "maximum": 255,
            "minimum": 0,
            "title": "B",
            "type": "integer"
        },
        "a": {
            "default": 255,
            "maximum": 255,
            "minimum": 0,
            "title": "A",
            "type": "integer"
        }
    },
    "required": [
        "r",
        "g",
        "b"
    ],
    "title": "ColorRGBA",
    "type": "object"
},
"ConfidenceReport": {
    "properties": {
        "parse_score": {
            "default": NaN,
            "title": "Parse Score",
            "type": "number"
        },
        "layout_score": {
            "default": NaN,
            "title": "Layout Score",
            "type": "number"
        }
    }
}

```

```

    },
    "table_score": {
      "default": NaN,
      "title": "Table Score",
      "type": "number"
    },
    "ocr_score": {
      "default": NaN,
      "title": "Ocr Score",
      "type": "number"
    },
    "pages": {
      "additionalProperties": {
        "$ref": "#/$defs/PageConfidenceScores"
      },
      "title": "Pages",
      "type": "object"
    }
  },
  "title": "ConfidenceReport",
  "type": "object"
},
"ContainerElement": {
  "properties": {
    "label": {
      "$ref": "#/$defs/DocItemLabel"
    },
    "id": {
      "title": "Id",
      "type": "integer"
    },
    "page_no": {
      "title": "Page No",
      "type": "integer"
    },
    "cluster": {
      "$ref": "#/$defs/Cluster"
    },
    "text": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Text"
    }
  },
  "required": [
    "label",
    "id",
    "page_no",
    "cluster"
  ],
  "title": "ContainerElement",
  "type": "object"
},
"ContentLayer": {
  "description": "ContentLayer.",
  "enum": [
    "body",
    "furniture",
    "background",
    "invisible",
    "notes"
  ],

```



```

    "title": "ContentLayer",
    "type": "string"
  },
  "ConversionStatus": {
    "enum": [
      "pending",
      "started",
      "failure",
      "success",
      "partial_success",
      "skipped"
    ],
    "title": "ConversionStatus",
    "type": "string"
  },
  "Coord2D": {
    "maxItems": 2,
    "minItems": 2,
    "prefixItems": [
      {
        "title": "X",
        "type": "number"
      },
      {
        "title": "Y",
        "type": "number"
      }
    ],
    "type": "array"
  },
  "CoordOrigin": {
    "description": "CoordOrigin.",
    "enum": [
      "TOPLEFT",
      "BOTTOMLEFT"
    ],
    "title": "CoordOrigin",
    "type": "string"
  },
  "DeclarativeBackendOptions": {
    "description": "Default backend options for a declarative document backend.",
    "properties": {
      "enable_remote_fetch": {
        "default": false,
        "description": "Enable remote resource fetching.",
        "title": "Enable Remote Fetch",
        "type": "boolean"
      },
      "enable_local_fetch": {
        "default": false,
        "description": "Enable local resource fetching.",
        "title": "Enable Local Fetch",
        "type": "boolean"
      },
      "kind": {
        "const": "declarative",
        "default": "declarative",
        "title": "Kind",
        "type": "string"
      }
    },
    "title": "DeclarativeBackendOptions",
    "type": "object"
  },
  "DescriptionAnnotation": {
    "description": "DescriptionAnnotation.",
    "properties": {
      "kind": {
        "const": "description",

```

```
        "default": "description",
        "title": "Kind",
        "type": "string"
    },
    "text": {
        "title": "Text",
        "type": "string"
    },
    "provenance": {
        "title": "Provenance",
        "type": "string"
    }
},
"required": [
    "text",
    "provenance"
],
"title": "DescriptionAnnotation",
"type": "object"
},
"DescriptionMetaField": {
    "additionalProperties": true,
    "description": "Description metadata field.",
    "properties": {
        "confidence": {
            "anyOf": [
                {
                    "maximum": 1,
                    "minimum": 0,
                    "type": "number"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "description": "The confidence of the prediction.",
            "examples": [
                0.9,
                0.42
            ],
            "title": "Confidence"
        },
        "created_by": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "description": "The origin of the prediction.",
            "examples": [
                "ibm-granite/granite-docling-258M"
            ],
            "title": "Created By"
        },
        "text": {
            "title": "Text",
            "type": "string"
        }
    },
    "required": [
        "text"
    ],
    "title": "DescriptionMetaField",
    "type": "object"
}
```

```

    },
    "DocItemLabel": {
      "description": "DocItemLabel.",
      "enum": [
        "caption",
        "chart",
        "footnote",
        "formula",
        "list_item",
        "page_footer",
        "page_header",
        "picture",
        "section_header",
        "table",
        "text",
        "title",
        "document_index",
        "code",
        "checkbox_selected",
        "checkbox_unselected",
        "form",
        "key_value_region",
        "grading_scale",
        "handwritten_text",
        "empty_value",
        "paragraph",
        "reference"
      ],
      "title": "DocItemLabel",
      "type": "string"
    },
    "DoclingComponentType": {
      "enum": [
        "document_backend",
        "model",
        "doc_assembler",
        "user_input",
        "pipeline"
      ],
      "title": "DoclingComponentType",
      "type": "string"
    },
    "DoclingDocument": {
      "description": "DoclingDocument.",
      "properties": {
        "schema_name": {
          "const": "DoclingDocument",
          "default": "DoclingDocument",
          "title": "Schema Name",
          "type": "string"
        },
        "version": {
          "default": "1.9.0",
          "pattern": "^(?P<major>0|[1-9]\\d*)\\.?(?P<minor>0|[1-9]\\d*)\\.?(?P<patch>0|[1-9]\\d*)(?:-(?P<prerelease>(?:0|[1-9]\\d*|\\d*[a-zA-Z-][0-9a-zA-Z-]*)|(?:\\.(?:0|[1-9]\\d*|\\d*[a-zA-Z-][0-9a-zA-Z-]*)*))?)?(?:\\+(?P<buildmetadata>[0-9a-zA-Z-]+(?:\\.\\d*[a-zA-Z-][0-9a-zA-Z-]*)*))?$",
          "title": "Version",
          "type": "string"
        },
        "name": {
          "title": "Name",
          "type": "string"
        },
        "origin": {
          "anyOf": [
            {
              "$ref": "#/$defs/DocumentOrigin"
            },
            {

```

```

        "type": "null"
    },
    ],
    "default": null
},
"furniture": {
    "$ref": "#/$defs/GroupItem",
    "default": {
        "self_ref": "#/furniture",
        "parent": null,
        "children": [],
        "content_layer": "furniture",
        "meta": null,
        "name": "_root_",
        "label": "unspecified"
    },
    "deprecated": true
},
"body": {
    "$ref": "#/$defs/GroupItem",
    "default": {
        "self_ref": "#/body",
        "parent": null,
        "children": [],
        "content_layer": "body",
        "meta": null,
        "name": "_root_",
        "label": "unspecified"
    }
},
"groups": {
    "default": [],
    "items": {
        "anyOf": [
            {
                "$ref": "#/$defs/ListGroup"
            },
            {
                "$ref": "#/$defs/InlineGroup"
            },
            {
                "$ref": "#/$defs/GroupItem"
            }
        ]
    },
    "title": "Groups",
    "type": "array"
},
"texts": {
    "default": [],
    "items": {
        "anyOf": [
            {
                "$ref": "#/$defs/TitleItem"
            },
            {
                "$ref": "#/$defs/SectionHeaderItem"
            },
            {
                "$ref": "#/$defs/ListItem"
            },
            {
                "$ref": "#/$defs/CodeItem"
            },
            {
                "$ref": "#/$defs/FormulaItem"
            },
            {
                "$ref": "#/$defs/TextItem"
            }
        ]
    }
}

```

```

    },
    "title": "Texts",
    "type": "array"
  },
  "pictures": {
    "default": [],
    "items": {
      "$ref": "#/$defs/PictureItem"
    },
    "title": "Pictures",
    "type": "array"
  },
  "tables": {
    "default": [],
    "items": {
      "$ref": "#/$defs/TableItem"
    },
    "title": "Tables",
    "type": "array"
  },
  "key_value_items": {
    "default": [],
    "items": {
      "$ref": "#/$defs/KeyValueItem"
    },
    "title": "Key Value Items",
    "type": "array"
  },
  "form_items": {
    "default": [],
    "items": {
      "$ref": "#/$defs/FormItem"
    },
    "title": "Form Items",
    "type": "array"
  },
  "pages": {
    "additionalProperties": {
      "$ref": "#/$defs/PageItem"
    },
    "default": {},
    "title": "Pages",
    "type": "object"
  }
},
"required": [
  "name"
],
"title": "DoclingDocument",
"type": "object"
},
"DoclingVersion": {
  "properties": {
    "docling_version": {
      "default": "2.80.0",
      "title": "Docling Version",
      "type": "string"
    },
    "docling_core_version": {
      "default": "2.69.0",
      "title": "Docling Core Version",
      "type": "string"
    },
    "docling_ibm_models_version": {
      "default": "3.12.0",
      "title": "Docling Ibm Models Version",
      "type": "string"
    }
  }
}

```

```

    },
    "docling_parse_version": {
      "default": "5.5.0",
      "title": "Docling Parse Version",
      "type": "string"
    },
    "platform_str": {
      "default": "Linux-6.14.0-1017-azure-x86_64-with-glibc2.39",
      "title": "Platform Str",
      "type": "string"
    },
    "py_impl_version": {
      "default": "cpython-312",
      "title": "Py Impl Version",
      "type": "string"
    },
    "py_lang_version": {
      "default": "3.12.3",
      "title": "Py Lang Version",
      "type": "string"
    }
  },
  "title": "DoclingVersion",
  "type": "object"
},
"DocumentLimits": {
  "properties": {
    "max_num_pages": {
      "default": 9223372036854775807,
      "title": "Max Num Pages",
      "type": "integer"
    },
    "max_file_size": {
      "default": 9223372036854775807,
      "title": "Max File Size",
      "type": "integer"
    },
    "page_range": {
      "default": [
        1,
        9223372036854775807
      ],
      "title": "Page Range"
    }
  },
  "title": "DocumentLimits",
  "type": "object"
},
"DocumentOrigin": {
  "description": "FileSource.",
  "properties": {
    "mimetype": {
      "title": "Mimetype",
      "type": "string"
    },
    "binary_hash": {
      "maximum": 18446744073709551615,
      "minimum": 0,
      "title": "Binary Hash",
      "type": "integer"
    },
    "filename": {
      "title": "Filename",
      "type": "string"
    },
    "uri": {
      "anyOf": [
        {
          "format": "uri",

```

```

        "minLength": 1,
        "type": "string"
    },
    {
        "type": "null"
    }
],
"default": null,
"title": "Uri"
}
},
"required": [
    "mimetype",
    "binary_hash",
    "filename"
],
"title": "DocumentOrigin",
"type": "object"
},
"EquationPrediction": {
    "properties": {
        "equation_count": {
            "default": 0,
            "title": "Equation Count",
            "type": "integer"
        },
        "equation_map": {
            "additionalProperties": {
                "$ref": "#/$defs/TextElement"
            },
            "default": {},
            "title": "Equation Map",
            "type": "object"
        }
    },
    "title": "EquationPrediction",
    "type": "object"
},
"ErrorItem": {
    "properties": {
        "component_type": {
            "$ref": "#/$defs/DoclingComponentType"
        },
        "module_name": {
            "title": "Module Name",
            "type": "string"
        },
        "error_message": {
            "title": "Error Message",
            "type": "string"
        }
    },
    "required": [
        "component_type",
        "module_name",
        "error_message"
    ],
    "title": "ErrorItem",
    "type": "object"
},
"FigureClassificationPrediction": {
    "properties": {
        "figure_count": {
            "default": 0,
            "title": "Figure Count",
            "type": "integer"
        },
        "figure_map": {
            "additionalProperties": {

```

```

        "$ref": "#/$defs/FigureElement"
    },
    "default": {},
    "title": "Figure Map",
    "type": "object"
  }
},
"title": "FigureClassificationPrediction",
"type": "object"
},
"FigureElement": {
  "properties": {
    "label": {
      "$ref": "#/$defs/DocItemLabel"
    },
    "id": {
      "title": "Id",
      "type": "integer"
    },
    "page_no": {
      "title": "Page No",
      "type": "integer"
    },
    "cluster": {
      "$ref": "#/$defs/Cluster"
    },
    "text": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Text"
    },
    "annotations": {
      "default": [],
      "items": {
        "discriminator": {
          "mapping": {
            "bar_chart_data": "#/$defs/PictureBarChartData",
            "classification": "#/$defs/PictureClassificationData",
            "description": "#/$defs/DescriptionAnnotation",
            "line_chart_data": "#/$defs/PictureLineChartData",
            "misc": "#/$defs/MiscAnnotation",
            "molecule_data": "#/$defs/PictureMoleculeData",
            "pie_chart_data": "#/$defs/PicturePieChartData",
            "scatter_chart_data": "#/$defs/PictureScatterChartData",
            "stacked_bar_chart_data": "#/$defs/PictureStackedBarChartData",
            "tabular_chart_data": "#/$defs/PictureTabularChartData"
          }
        },
        "propertyName": "kind"
      },
      "oneOf": [
        {
          "$ref": "#/$defs/DescriptionAnnotation"
        },
        {
          "$ref": "#/$defs/MiscAnnotation"
        },
        {
          "$ref": "#/$defs/PictureClassificationData"
        },
        {
          "$ref": "#/$defs/PictureMoleculeData"
        }
      ]
    }
  }
}

```



```

        {
            "$ref": "#/$defs/PictureTabularChartData"
        },
        {
            "$ref": "#/$defs/PictureLineChartData"
        },
        {
            "$ref": "#/$defs/PictureBarChartData"
        },
        {
            "$ref": "#/$defs/PictureStackedBarChartData"
        },
        {
            "$ref": "#/$defs/PicturePieChartData"
        },
        {
            "$ref": "#/$defs/PictureScatterChartData"
        }
    ],
    "title": "Annotations",
    "type": "array"
},
"provenance": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Provenance"
},
"predicted_class": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Predicted Class"
},
"confidence": {
    "anyOf": [
        {
            "type": "number"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Confidence"
}
},
"required": [
    "label",
    "id",
    "page_no",
    "cluster"
],
"title": "FigureElement",
"type": "object"
},

```

```

    "FineRef": {
      "description": "Fine-granular reference item that can capture span range info.",
      "properties": {
        "$ref": {
          "pattern": "^(?:(/[\\w-]+)(?:/(\\d+))?)?$",
          "title": "$Ref",
          "type": "string"
        },
        "range": {
          "anyOf": [
            {
              "maxItems": 2,
              "minItems": 2,
              "prefixItems": [
                {
                  "type": "integer"
                },
                {
                  "type": "integer"
                }
              ],
              "type": "array"
            },
            {
              "type": "null"
            }
          ],
          "default": null,
          "title": "Range"
        }
      },
      "required": [
        "$ref"
      ],
      "title": "FineRef",
      "type": "object"
    },
    "FloatingMeta": {
      "additionalProperties": true,
      "description": "Metadata model for floating.",
      "properties": {
        "summary": {
          "anyOf": [
            {
              "$ref": "#/$defs/SummaryMetaField"
            },
            {
              "type": "null"
            }
          ],
          "default": null
        },
        "description": {
          "anyOf": [
            {
              "$ref": "#/$defs/DescriptionMetaField"
            },
            {
              "type": "null"
            }
          ],
          "default": null
        }
      },
      "title": "FloatingMeta",
      "type": "object"
    },
    "FormItem": {
      "additionalProperties": false,

```

```
"description": "FormItem.",
"properties": {
  "self_ref": {
    "pattern": "^#(?:/[\\w-]+)(?:/(\\d+))?$",
    "title": "Self Ref",
    "type": "string"
  },
  "parent": {
    "anyOf": [
      {
        "$ref": "#/$defs/RefItem"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "children": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Children",
    "type": "array"
  },
  "content_layer": {
    "$ref": "#/$defs/ContentLayer",
    "default": "body"
  },
  "meta": {
    "anyOf": [
      {
        "$ref": "#/$defs/FloatingMeta"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "label": {
    "const": "form",
    "default": "form",
    "title": "Label",
    "type": "string"
  },
  "prov": {
    "default": [],
    "items": {
      "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
  },
  "source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
    "items": {
      "discriminator": {
        "mapping": {
          "track": "#/$defs/TrackSource"
        }
      },
      "propertyName": "kind"
    },
    "oneOf": [
      {
        "$ref": "#/$defs/TrackSource"
```

```

    },
    "title": "Source",
    "type": "array"
  },
  "comments": {
    "default": [],
    "items": {
      "$ref": "#/$defs/FineRef"
    },
    "title": "Comments",
    "type": "array"
  },
  "captions": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Captions",
    "type": "array"
  },
  "references": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "References",
    "type": "array"
  },
  "footnotes": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Footnotes",
    "type": "array"
  },
  "image": {
    "anyOf": [
      {
        "$ref": "#/$defs/ImageRef"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "graph": {
    "$ref": "#/$defs/GraphData"
  }
},
"required": [
  "self_ref",
  "graph"
],
"title": "FormItem",
"type": "object"
},
"Formatting": {
  "description": "Formatting.",
  "properties": {
    "bold": {
      "default": false,
      "title": "Bold",
      "type": "boolean"
    },
    "italic": {

```

```

        "default": false,
        "title": "Italic",
        "type": "boolean"
    },
    "underline": {
        "default": false,
        "title": "Underline",
        "type": "boolean"
    },
    "strikethrough": {
        "default": false,
        "title": "Strikethrough",
        "type": "boolean"
    },
    "script": {
        "$ref": "#/$defs/Script",
        "default": "baseline"
    }
},
"title": "Formatting",
"type": "object"
},
"FormulaItem": {
    "additionalProperties": false,
    "description": "FormulaItem.",
    "properties": {
        "self_ref": {
            "pattern": "^#(?:/(\\w-)+)?(?:/(\\d+)))?$ ",
            "title": "Self Ref",
            "type": "string"
        },
        "parent": {
            "anyOf": [
                {
                    "$ref": "#/$defs/RefItem"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "children": {
            "default": [],
            "items": {
                "$ref": "#/$defs/RefItem"
            },
            "title": "Children",
            "type": "array"
        },
        "content_layer": {
            "$ref": "#/$defs/ContentLayer",
            "default": "body"
        },
        "meta": {
            "anyOf": [
                {
                    "$ref": "#/$defs/BaseMeta"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "label": {
            "const": "formula",
            "default": "formula",
            "title": "Label",

```

```
    "type": "string"
  },
  "prov": {
    "default": [],
    "items": {
      "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
  },
  "source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
    "items": {
      "discriminator": {
        "mapping": {
          "track": "#/$defs/TrackSource"
        }
      },
      "propertyName": "kind"
    },
    "oneOf": [
      {
        "$ref": "#/$defs/TrackSource"
      }
    ]
  },
  "title": "Source",
  "type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"orig": {
  "title": "Orig",
  "type": "string"
},
"text": {
  "title": "Text",
  "type": "string"
},
"formatting": {
  "anyOf": [
    {
      "$ref": "#/$defs/Formatting"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"hyperlink": {
  "anyOf": [
    {
      "format": "uri",
      "minLength": 1,
      "type": "string"
    },
    {
      "format": "path",
      "type": "string"
    }
  ],
  {
```

```

        "type": "null"
    },
    ],
    "default": null,
    "title": "Hyperlink"
},
},
"required": [
    "self_ref",
    "orig",
    "text"
],
"title": "FormulaItem",
"type": "object"
},
"GraphCell": {
    "description": "GraphCell.",
    "properties": {
        "label": {
            "$ref": "#/$defs/GraphCellLabel"
        },
        "cell_id": {
            "title": "Cell Id",
            "type": "integer"
        },
        "text": {
            "title": "Text",
            "type": "string"
        },
        "orig": {
            "title": "Orig",
            "type": "string"
        },
        "prov": {
            "anyOf": [
                {
                    "$ref": "#/$defs/ProvenanceItem"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "item_ref": {
            "anyOf": [
                {
                    "$ref": "#/$defs/RefItem"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        }
    },
    "required": [
        "label",
        "cell_id",
        "text",
        "orig"
    ],
    "title": "GraphCell",
    "type": "object"
},
"GraphCellLabel": {
    "description": "GraphCellLabel.",
    "enum": [
        "unspecified",

```

```

        "key",
        "value",
        "checkbox"
    ],
    "title": "GraphCellLabel",
    "type": "string"
},
"GraphData": {
    "description": "GraphData.",
    "properties": {
        "cells": {
            "items": {
                "$ref": "#/$defs/GraphCell"
            },
            "title": "Cells",
            "type": "array"
        },
        "links": {
            "items": {
                "$ref": "#/$defs/GraphLink"
            },
            "title": "Links",
            "type": "array"
        }
    },
    "title": "GraphData",
    "type": "object"
},
"GraphLink": {
    "description": "GraphLink.",
    "properties": {
        "label": {
            "$ref": "#/$defs/GraphLinkLabel"
        },
        "source_cell_id": {
            "title": "Source Cell Id",
            "type": "integer"
        },
        "target_cell_id": {
            "title": "Target Cell Id",
            "type": "integer"
        }
    },
    "required": [
        "label",
        "source_cell_id",
        "target_cell_id"
    ],
    "title": "GraphLink",
    "type": "object"
},
"GraphLinkLabel": {
    "description": "GraphLinkLabel.",
    "enum": [
        "unspecified",
        "to_value",
        "to_key",
        "to_parent",
        "to_child"
    ],
    "title": "GraphLinkLabel",
    "type": "string"
},
"GroupItem": {
    "additionalProperties": false,
    "description": "GroupItem.",
    "properties": {
        "self_ref": {
            "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$"
        }
    }
}

```



```

        "title": "Self Ref",
        "type": "string"
    },
    "parent": {
        "anyOf": [
            {
                "$ref": "#/$defs/RefItem"
            },
            {
                "type": "null"
            }
        ],
        "default": null
    },
    "children": {
        "default": [],
        "items": {
            "$ref": "#/$defs/RefItem"
        },
        "title": "Children",
        "type": "array"
    },
    "content_layer": {
        "$ref": "#/$defs/ContentLayer",
        "default": "body"
    },
    "meta": {
        "anyOf": [
            {
                "$ref": "#/$defs/BaseMeta"
            },
            {
                "type": "null"
            }
        ],
        "default": null
    },
    "name": {
        "default": "group",
        "title": "Name",
        "type": "string"
    },
    "label": {
        "$ref": "#/$defs/GroupLabel",
        "default": "unspecified"
    }
},
"required": [
    "self_ref"
],
"title": "GroupItem",
"type": "object"
},
"GroupLabel": {
    "description": "GroupLabel.",
    "enum": [
        "unspecified",
        "list",
        "ordered_list",
        "chapter",
        "section",
        "sheet",
        "slide",
        "form_area",
        "key_value_area",
        "comment_section",
        "inline",
        "picture_area"
    ],

```

```

    "title": "GroupLabel",
    "type": "string"
  },
  "HTMLBackendOptions": {
    "description": "Options specific to the HTML backend.\n\nThis class can be extended to include options specific to HTML processing.",
    "properties": {
      "enable_remote_fetch": {
        "default": false,
        "description": "Enable remote resource fetching.",
        "title": "Enable Remote Fetch",
        "type": "boolean"
      },
      "enable_local_fetch": {
        "default": false,
        "description": "Enable local resource fetching.",
        "title": "Enable Local Fetch",
        "type": "boolean"
      },
      "kind": {
        "const": "html",
        "default": "html",
        "title": "Kind",
        "type": "string"
      },
      "fetch_images": {
        "default": false,
        "description": "Whether the backend should access remote or local resources to parse images in an HTML document.",
        "title": "Fetch Images",
        "type": "boolean"
      },
      "source_uri": {
        "anyOf": [
          {
            "format": "uri",
            "minLength": 1,
            "type": "string"
          },
          {
            "format": "path",
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The URI that originates the HTML document. If provided, the backend will use it to resolve relative paths in the HTML document.",
        "title": "Source Uri"
      },
      "add_title": {
        "default": true,
        "description": "Add the HTML title tag as furniture in the DoclingDocument.",
        "title": "Add Title",
        "type": "boolean"
      },
      "infer_furniture": {
        "default": true,
        "description": "Infer all the content before the first header as furniture.",
        "title": "Infer Furniture",
        "type": "boolean"
      }
    },
    "title": "HTMLBackendOptions",
    "type": "object"
  },
  "ImageRef": {

```

```

    "description": "ImageRef.",
    "properties": {
      "mimetype": {
        "title": "Mimetype",
        "type": "string"
      },
      "dpi": {
        "title": "Dpi",
        "type": "integer"
      },
      "size": {
        "$ref": "#/$defs/Size"
      },
      "uri": {
        "anyOf": [
          {
            "format": "uri",
            "minLength": 1,
            "type": "string"
          },
          {
            "format": "path",
            "type": "string"
          }
        ],
        "title": "Uri"
      }
    },
    "required": [
      "mimetype",
      "dpi",
      "size",
      "uri"
    ],
    "title": "ImageRef",
    "type": "object"
  },
  "ImageRefMode": {
    "description": "ImageRefMode.",
    "enum": [
      "placeholder",
      "embedded",
      "referenced"
    ],
    "title": "ImageRefMode",
    "type": "string"
  },
  "InlineGroup": {
    "additionalProperties": false,
    "description": "InlineGroup.",
    "properties": {
      "self_ref": {
        "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
        "title": "Self Ref",
        "type": "string"
      },
      "parent": {
        "anyOf": [
          {
            "$ref": "#/$defs/RefItem"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "children": {
        "default": [],

```

```

    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Children",
    "type": "array"
  },
  "content_layer": {
    "$ref": "#/$defs/ContentLayer",
    "default": "body"
  },
  "meta": {
    "anyOf": [
      {
        "$ref": "#/$defs/BaseMeta"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "name": {
    "default": "group",
    "title": "Name",
    "type": "string"
  },
  "label": {
    "const": "inline",
    "default": "inline",
    "title": "Label",
    "type": "string"
  }
},
"required": [
  "self_ref"
],
"title": "InlineGroup",
"type": "object"
},
"InputDocument": {
  "description": "A document as an input of a Docling conversion.",
  "properties": {
    "file": {
      "description": "A path representation the input document.",
      "format": "path",
      "title": "File",
      "type": "string"
    },
    "document_hash": {
      "description": "A stable hash of the path or stream of the input document.",
      "title": "Document Hash",
      "type": "string"
    },
    "valid": {
      "default": true,
      "description": "Whether this is is a valid input document.",
      "title": "Valid",
      "type": "boolean"
    },
    "backend_options": {
      "anyOf": [
        {
          "discriminator": {
            "mapping": {
              "declarative": "#/$defs/DeclarativeBackendOptions",
              "html": "#/$defs/HTMLBackendOptions",
              "latex": "#/$defs/LatexBackendOptions",
              "md": "#/$defs/MarkdownBackendOptions",
              "pdf": "#/$defs/PdfBackendOptions",

```

```

        "xbrl": "#/$defs/XBRLBackendOptions",
        "xlsx": "#/$defs/MsExcelBackendOptions"
    },
    "propertyName": "kind"
},
"oneOf": [
    {
        "$ref": "#/$defs/DeclarativeBackendOptions"
    },
    {
        "$ref": "#/$defs/HTMLBackendOptions"
    },
    {
        "$ref": "#/$defs/MarkdownBackendOptions"
    },
    {
        "$ref": "#/$defs/PdfBackendOptions"
    },
    {
        "$ref": "#/$defs/MsExcelBackendOptions"
    },
    {
        "$ref": "#/$defs/LatexBackendOptions"
    },
    {
        "$ref": "#/$defs/XBRLBackendOptions"
    }
],
{
    "type": "null"
}
],
"default": null,
"description": "Custom options for backends.",
"title": "Backend Options"
},
"limits": {
    "$ref": "#/$defs/DocumentLimits",
    "default": {
        "max_num_pages": 9223372036854775807,
        "max_file_size": 9223372036854775807,
        "page_range": [
            1,
            9223372036854775807
        ]
    },
    "description": "Limits in the input document for the conversion."
},
"format": {
    "$ref": "#/$defs/InputFormat",
    "description": "The document format."
},
"filesize": {
    "anyOf": [
        {
            "type": "integer"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "description": "Size of the input file, in bytes.",
    "title": "Filesize"
},
"page_count": {
    "default": 0,
    "description": "Number of pages in the input document.",

```

```

        "title": "Page Count",
        "type": "integer"
    },
    },
    "required": [
        "file",
        "document_hash",
        "format"
    ],
    "title": "InputDocument",
    "type": "object"
},
"InputFormat": {
    "description": "A document format supported by document backend parsers.",
    "enum": [
        "docx",
        "pptx",
        "html",
        "image",
        "pdf",
        "asciidoc",
        "md",
        "csv",
        "xlsx",
        "xml_uspto",
        "xml_jats",
        "xml_xbrl",
        "mets_gbs",
        "json_docling",
        "audio",
        "vtt",
        "latex"
    ],
    "title": "InputFormat",
    "type": "string"
},
"KeyValueItem": {
    "additionalProperties": false,
    "description": "KeyValueItem.",
    "properties": {
        "self_ref": {
            "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
            "title": "Self Ref",
            "type": "string"
        },
    },
    "parent": {
        "anyOf": [
            {
                "$ref": "#/$defs/RefItem"
            },
            {
                "type": "null"
            }
        ],
    },
    "default": null
},
"children": {
    "default": [],
    "items": {
        "$ref": "#/$defs/RefItem"
    },
    "title": "Children",
    "type": "array"
},
"content_layer": {
    "$ref": "#/$defs/ContentLayer",
    "default": "body"
},
"meta": {

```

```

    "anyOf": [
      {
        "$ref": "#/$defs/FloatingMeta"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "label": {
    "const": "key_value_region",
    "default": "key_value_region",
    "title": "Label",
    "type": "string"
  },
  "prov": {
    "default": [],
    "items": {
      "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
  },
  "source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
    "items": {
      "discriminator": {
        "mapping": {
          "track": "#/$defs/TrackSource"
        }
      },
      "propertyName": "kind"
    },
    "oneOf": [
      {
        "$ref": "#/$defs/TrackSource"
      }
    ]
  },
  "title": "Source",
  "type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"captions": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "Captions",
  "type": "array"
},
"references": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "References",
  "type": "array"
},
"footnotes": {

```

```

    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Footnotes",
    "type": "array"
  },
  "image": {
    "anyOf": [
      {
        "$ref": "#/$defs/ImageRef"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "graph": {
    "$ref": "#/$defs/GraphData"
  }
},
"required": [
  "self_ref",
  "graph"
],
"title": "KeyValueItem",
"type": "object"
},
"LatexBackendOptions": {
  "description": "Options specific to the LaTeX backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "latex",
      "default": "latex",
      "title": "Kind",
      "type": "string"
    },
    "parse_timeout": {
      "anyOf": [
        {
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": 30.0,
      "description": "Maximum time allowed for parsing a LaTeX document. Set to None to disable the timeout. Defaults to 30 s.",
      "title": "Parse Timeout"
    }
  },
  "title": "LatexBackendOptions",
  "type": "object"
},
"LayoutPrediction": {

```



```

    "properties": {
      "clusters": {
        "default": [],
        "items": {
          "$ref": "#/$defs/Cluster"
        },
        "title": "Clusters",
        "type": "array"
      }
    },
    "title": "LayoutPrediction",
    "type": "object"
  },
  "ListGroup": {
    "additionalProperties": false,
    "description": "ListGroup.",
    "properties": {
      "self_ref": {
        "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
        "title": "Self Ref",
        "type": "string"
      },
      "parent": {
        "anyOf": [
          {
            "$ref": "#/$defs/RefItem"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "children": {
        "default": [],
        "items": {
          "$ref": "#/$defs/RefItem"
        },
        "title": "Children",
        "type": "array"
      },
      "content_layer": {
        "$ref": "#/$defs/ContentLayer",
        "default": "body"
      },
      "meta": {
        "anyOf": [
          {
            "$ref": "#/$defs/BaseMeta"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "name": {
        "default": "group",
        "title": "Name",
        "type": "string"
      },
      "label": {
        "const": "list",
        "default": "list",
        "title": "Label",
        "type": "string"
      }
    },
    "required": [

```

```

    "self_ref": {
    },
    "title": "ListGroup",
    "type": "object"
  },
  "ListItem": {
    "additionalProperties": false,
    "description": "SectionItem.",
    "properties": {
      "self_ref": {
        "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
        "title": "Self Ref",
        "type": "string"
      },
      "parent": {
        "anyOf": [
          {
            "$ref": "#/$defs/RefItem"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "children": {
        "default": [],
        "items": {
          "$ref": "#/$defs/RefItem"
        },
        "title": "Children",
        "type": "array"
      },
      "content_layer": {
        "$ref": "#/$defs/ContentLayer",
        "default": "body"
      },
      "meta": {
        "anyOf": [
          {
            "$ref": "#/$defs/BaseMeta"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "label": {
        "const": "list_item",
        "default": "list_item",
        "title": "Label",
        "type": "string"
      },
      "prov": {
        "default": [],
        "items": {
          "$ref": "#/$defs/ProvenanceItem"
        },
        "title": "Prov",
        "type": "array"
      },
      "source": {
        "default": [],
        "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
        "items": {
          "discriminator": {
            "mapping": {

```

```

    "track": "#/$defs/TrackSource"
  },
  "propertyName": "kind"
},
"oneOf": [
  {
    "$ref": "#/$defs/TrackSource"
  }
]
},
"title": "Source",
"type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"orig": {
  "title": "Orig",
  "type": "string"
},
"text": {
  "title": "Text",
  "type": "string"
},
"formatting": {
  "anyOf": [
    {
      "$ref": "#/$defs/Formatting"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"hyperlink": {
  "anyOf": [
    {
      "format": "uri",
      "minLength": 1,
      "type": "string"
    },
    {
      "format": "path",
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Hyperlink"
},
"enumerated": {
  "default": false,
  "title": "Enumerated",
  "type": "boolean"
},
"marker": {
  "default": "-",
  "title": "Marker",
  "type": "string"
}
},

```

```

    "required": [
      "self_ref",
      "orig",
      "text"
    ],
    "title": "ListItem",
    "type": "object"
  },
  "MarkdownBackendOptions": {
    "description": "Options specific to the Markdown backend.",
    "properties": {
      "enable_remote_fetch": {
        "default": false,
        "description": "Enable remote resource fetching.",
        "title": "Enable Remote Fetch",
        "type": "boolean"
      },
      "enable_local_fetch": {
        "default": false,
        "description": "Enable local resource fetching.",
        "title": "Enable Local Fetch",
        "type": "boolean"
      },
      "kind": {
        "const": "md",
        "default": "md",
        "title": "Kind",
        "type": "string"
      },
      "fetch_images": {
        "default": false,
        "description": "Whether the backend should access remote or local resources to parse images in the markdown document.",
        "title": "Fetch Images",
        "type": "boolean"
      },
      "source_uri": {
        "anyOf": [
          {
            "format": "uri",
            "minLength": 1,
            "type": "string"
          },
          {
            "format": "path",
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The URI that originates the markdown document. If provided, the backend will use it to resolve relative paths in the markdown document.",
        "title": "Source Uri"
      }
    },
    "title": "MarkdownBackendOptions",
    "type": "object"
  },
  "MiscAnnotation": {
    "description": "MiscAnnotation.",
    "properties": {
      "kind": {
        "const": "misc",
        "default": "misc",
        "title": "Kind",
        "type": "string"
      }
    },

```

```
    "content": {
      "additionalProperties": true,
      "title": "Content",
      "type": "object"
    }
  },
  "required": [
    "content"
  ],
  "title": "MiscAnnotation",
  "type": "object"
},
"MoleculeMetaField": {
  "additionalProperties": true,
  "description": "Molecule metadata field.",
  "properties": {
    "confidence": {
      "anyOf": [
        {
          "maximum": 1,
          "minimum": 0,
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The confidence of the prediction.",
      "examples": [
        0.9,
        0.42
      ],
      "title": "Confidence"
    },
    "created_by": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The origin of the prediction.",
      "examples": [
        "ibm-granite/granite-docling-258M"
      ],
      "title": "Created By"
    },
    "smi": {
      "description": "The SMILES representation of the molecule.",
      "title": "Smi",
      "type": "string"
    }
  },
  "required": [
    "smi"
  ],
  "title": "MoleculeMetaField",
  "type": "object"
},
"MsExcelBackendOptions": {
  "description": "Options specific to the MS Excel backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
```

```

        "title": "Enable Remote Fetch",
        "type": "boolean"
    },
    "enable_local_fetch": {
        "default": false,
        "description": "Enable local resource fetching.",
        "title": "Enable Local Fetch",
        "type": "boolean"
    },
    "kind": {
        "const": "xlsx",
        "default": "xlsx",
        "title": "Kind",
        "type": "string"
    },
    "treat_singleton_as_text": {
        "default": false,
        "description": "Whether to treat singleton cells (1x1 tables with empty neighboring cells) as TextItem instead of TableItem.",
        "title": "Treat Singleton As Text",
        "type": "boolean"
    },
    "gap_tolerance": {
        "default": 0,
        "description": "The tolerance (in number of empty rows/columns) for merging nearby data clusters into a single table. Default is 0 (strict).",
        "title": "Gap Tolerance",
        "type": "integer"
    }
},
"title": "MsExcelBackendOptions",
"type": "object"
},
"Page": {
    "properties": {
        "page_no": {
            "title": "Page No",
            "type": "integer"
        },
        "size": {
            "anyOf": [
                {
                    "$ref": "#/$defs/Size"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "parsed_page": {
            "anyOf": [
                {
                    "$ref": "#/$defs/SegmentedPdfPage"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "predictions": {
            "$ref": "#/$defs/PagePredictions",
            "default": {
                "layout": null,
                "tablestructure": null,
                "figures_classification": null,
                "equations_prediction": null,
                "vlm_response": null
            }
        }
    }
}

```

```

    },
    "assembled": {
      "anyOf": [
        {
          "$ref": "#/$defs/AssembledUnit"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    }
  },
  "required": [
    "page_no"
  ],
  "title": "Page",
  "type": "object"
},
"PageConfidenceScores": {
  "properties": {
    "parse_score": {
      "default": NaN,
      "title": "Parse Score",
      "type": "number"
    },
    "layout_score": {
      "default": NaN,
      "title": "Layout Score",
      "type": "number"
    },
    "table_score": {
      "default": NaN,
      "title": "Table Score",
      "type": "number"
    },
    "ocr_score": {
      "default": NaN,
      "title": "Ocr Score",
      "type": "number"
    }
  },
  "title": "PageConfidenceScores",
  "type": "object"
},
"PageItem": {
  "description": "PageItem.",
  "properties": {
    "size": {
      "$ref": "#/$defs/Size"
    },
    "image": {
      "anyOf": [
        {
          "$ref": "#/$defs/ImageRef"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "page_no": {
      "title": "Page No",
      "type": "integer"
    }
  },
  "required": [

```

```

        "size": 1,
        "page_no": 1,
    },
    "title": "PageItem",
    "type": "object"
},
"PagePredictions": {
    "properties": {
        "layout": {
            "anyOf": [
                {
                    "$ref": "#/$defs/LayoutPrediction"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "tablestructure": {
            "anyOf": [
                {
                    "$ref": "#/$defs/TableStructurePrediction"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "figures_classification": {
            "anyOf": [
                {
                    "$ref": "#/$defs/FigureClassificationPrediction"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "equations_prediction": {
            "anyOf": [
                {
                    "$ref": "#/$defs/EquationPrediction"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "vlm_response": {
            "anyOf": [
                {
                    "$ref": "#/$defs/VlmPrediction"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        }
    },
    "title": "PagePredictions",
    "type": "object"
},
"PdfBackendOptions": {
    "description": "Backend options for pdf document backends.",

```



```

"properties": {
  "enable_remote_fetch": {
    "default": false,
    "description": "Enable remote resource fetching.",
    "title": "Enable Remote Fetch",
    "type": "boolean"
  },
  "enable_local_fetch": {
    "default": false,
    "description": "Enable local resource fetching.",
    "title": "Enable Local Fetch",
    "type": "boolean"
  },
  "kind": {
    "const": "pdf",
    "default": "pdf",
    "title": "Kind",
    "type": "string"
  },
  "password": {
    "anyOf": [
      {
        "format": "password",
        "type": "string",
        "writeOnly": true
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Password"
  }
},
"title": "PdfBackendOptions",
"type": "object"
},
"PdfCellRenderingMode": {
  "description": "Text Rendering Mode, according to PDF32000.",
  "enum": [
    0
    1
    2
    3
    4
    5
    6
    7
    -1
  ],
  "title": "PdfCellRenderingMode",
  "type": "integer"
},
"PdfHyperlink": {
  "properties": {
    "index": {
      "default": -1,
      "title": "Index",
      "type": "integer"
    },
    "rect": {
      "$ref": "#/$defs/BoundingBox"
    },
    "uri": {
      "anyOf": [
        {
          "format": "uri",
          "minLength": 1,
          "type": "string"
        }
      ]
    }
  }
}

```

```

    },
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Uri"
},
"widget_text": {
  "anyOf": [
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Widget Text"
},
"widget_description": {
  "anyOf": [
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Widget Description"
}
},
"required": [
  "rect"
],
"title": "PdfHyperlink",
"type": "object"
},
"PdfLine": {
  "description": "Model representing a line in a PDF document.\n\n.. deprecated::\n    Use :class:`PdfShape` instead.",
  "properties": {
    "index": {
      "default": -1,
      "title": "Index",
      "type": "integer"
    },
    "rgba": {
      "$ref": "#/$defs/ColorRGBA",
      "default": {
        "r": 0,
        "g": 0,
        "b": 0,
        "a": 255
      }
    },
    "parent_id": {
      "title": "Parent Id",
      "type": "integer"
    },
    "points": {
      "items": {
        "$ref": "#/$defs/Coord2D"
      },
      "title": "Points",

```

```

    "type": "array"
  },
  "width": {
    "default": 1.0,
    "title": "Width",
    "type": "number"
  },
  "coord_origin": {
    "$ref": "#/$defs/CoordOrigin",
    "default": "BOTTOMLEFT"
  }
},
"required": [
  "parent_id",
  "points"
],
"title": "PdfLine",
"type": "object"
},
"PdfPageBoundaryType": {
  "description": "Enumeration of PDF page boundary types.",
  "enum": [
    "art_box",
    "bleed_box",
    "crop_box",
    "media_box",
    "trim_box"
  ],
  "title": "PdfPageBoundaryType",
  "type": "string"
},
"PdfPageGeometry": {
  "description": "Extended dimensions model specific to PDF pages with boundary types.",
  "properties": {
    "angle": {
      "title": "Angle",
      "type": "number"
    },
    "rect": {
      "$ref": "#/$defs/BoundingBox"
    },
    "boundary_type": {
      "$ref": "#/$defs/PdfPageBoundaryType"
    },
    "art_bbox": {
      "$ref": "#/$defs/BoundingBox"
    },
    "bleed_bbox": {
      "$ref": "#/$defs/BoundingBox"
    },
    "crop_bbox": {
      "$ref": "#/$defs/BoundingBox"
    },
    "media_bbox": {
      "$ref": "#/$defs/BoundingBox"
    },
    "trim_bbox": {
      "$ref": "#/$defs/BoundingBox"
    }
  },
  "required": [
    "angle",
    "rect",
    "boundary_type",
    "art_bbox",
    "bleed_bbox",
    "crop_bbox",
    "media_bbox",
    "trim_bbox"
  ]
}

```

```

    },
    "title": "PdfPageGeometry",
    "type": "object"
  },
  "PdfShape": {
    "description": "Model representing a vector shape in a PDF document.",
    "properties": {
      "index": {
        "default": -1,
        "title": "Index",
        "type": "integer"
      },
      "parent_id": {
        "title": "Parent Id",
        "type": "integer"
      },
      "points": {
        "items": {
          "$ref": "#/$defs/Coord2D"
        },
        "title": "Points",
        "type": "array"
      },
      "coord_origin": {
        "$ref": "#/$defs/CoordOrigin",
        "default": "BOTTOMLEFT"
      },
      "has_graphics_state": {
        "default": false,
        "title": "Has Graphics State",
        "type": "boolean"
      },
      "line_width": {
        "default": -1.0,
        "title": "Line Width",
        "type": "number"
      },
      "miter_limit": {
        "default": -1.0,
        "title": "Miter Limit",
        "type": "number"
      },
      "line_cap": {
        "default": -1,
        "title": "Line Cap",
        "type": "integer"
      },
      "line_join": {
        "default": -1,
        "title": "Line Join",
        "type": "integer"
      },
      "dash_phase": {
        "default": 0.0,
        "title": "Dash Phase",
        "type": "number"
      },
      "dash_array": {
        "default": [],
        "items": {
          "type": "number"
        },
        "title": "Dash Array",
        "type": "array"
      },
      "flatness": {
        "default": -1.0,
        "title": "Flatness",
        "type": "number"
      }
    }
  }
}

```

```

    },
    "rgb_stroking": {
      "$ref": "#/$defs/ColorRGBA",
      "default": {
        "r": 0,
        "g": 0,
        "b": 0,
        "a": 255
      }
    },
    "rgb_filling": {
      "$ref": "#/$defs/ColorRGBA",
      "default": {
        "r": 0,
        "g": 0,
        "b": 0,
        "a": 255
      }
    },
    "rgba": {
      "anyOf": [
        {
          "$ref": "#/$defs/ColorRGBA"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "deprecated": true
    },
    "width": {
      "anyOf": [
        {
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "deprecated": true,
      "title": "Width"
    },
  },
  "required": [
    "parent_id",
    "points"
  ],
  "title": "PdfShape",
  "type": "object"
},
"PdfTextCell": {
  "description": "Specialized text cell for PDF documents with font information.",
  "properties": {
    "index": {
      "default": -1,
      "title": "Index",
      "type": "integer"
    },
    "rgba": {
      "$ref": "#/$defs/ColorRGBA",
      "default": {
        "r": 0,
        "g": 0,
        "b": 0,
        "a": 255
      }
    }
  },

```

```
    "rect": {
      "$ref": "#/$defs/BoundingBox"
    },
    "text": {
      "title": "Text",
      "type": "string"
    },
    "orig": {
      "title": "Orig",
      "type": "string"
    },
    "text_direction": {
      "$ref": "#/$defs/TextDirection",
      "default": "left_to_right"
    },
    "confidence": {
      "default": 1.0,
      "title": "Confidence",
      "type": "number"
    },
    "from_ocr": {
      "const": false,
      "default": false,
      "title": "From Ocr",
      "type": "boolean"
    },
    "rendering_mode": {
      "$ref": "#/$defs/PdfCellRenderingMode"
    },
    "widget": {
      "title": "Widget",
      "type": "boolean"
    },
    "font_key": {
      "title": "Font Key",
      "type": "string"
    },
    "font_name": {
      "title": "Font Name",
      "type": "string"
    }
  },
  "required": [
    "rect",
    "text",
    "orig",
    "rendering_mode",
    "widget",
    "font_key",
    "font_name"
  ],
  "title": "PdfTextCell",
  "type": "object"
},
"PdfWidget": {
  "properties": {
    "index": {
      "default": -1,
      "title": "Index",
      "type": "integer"
    },
    "rect": {
      "$ref": "#/$defs/BoundingBox"
    },
    "widget_text": {
      "anyOf": [
        {
          "type": "string"
        }
      ]
    }
  }
}
```

```

        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Widget Text"
},
"widget_description": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Widget Description"
},
"widget_field_name": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Widget Field Name"
},
"widget_field_type": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Widget Field Type"
}
},
"required": [
    "rect"
],
"title": "PdfWidget",
"type": "object"
},
"PictureBarChartData": {
    "description": "Represents data of a bar chart.\n\nAttributes:\n    kind (Literal[\"bar_chart_data\"]): The type of the chart.\n    x_axis_label (str): The label for the x-axis.\n    y_axis_label (str): The label for the y-axis.\n    bars (list[ChartBar]): A list of bars in the chart.",
    "properties": {
        "kind": {
            "const": "bar_chart_data",
            "default": "bar_chart_data",
            "title": "Kind",
            "type": "string"
        },
        "title": {
            "title": "Title",
            "type": "string"
        },
        "x_axis_label": {
            "title": "X Axis Label",
            "type": "string"
        },

```

```

    "y_axis_label": {
      "title": "Y Axis Label",
      "type": "string"
    },
    "bars": {
      "items": {
        "$ref": "#/$defs/ChartBar"
      },
      "title": "Bars",
      "type": "array"
    }
  },
  "required": [
    "title",
    "x_axis_label",
    "y_axis_label",
    "bars"
  ],
  "title": "PictureBarChartData",
  "type": "object"
},
"PictureClassificationClass": {
  "description": "PictureClassificationData.",
  "properties": {
    "class_name": {
      "title": "Class Name",
      "type": "string"
    },
    "confidence": {
      "title": "Confidence",
      "type": "number"
    }
  },
  "required": [
    "class_name",
    "confidence"
  ],
  "title": "PictureClassificationClass",
  "type": "object"
},
"PictureClassificationData": {
  "description": "PictureClassificationData.",
  "properties": {
    "kind": {
      "const": "classification",
      "default": "classification",
      "title": "Kind",
      "type": "string"
    },
    "provenance": {
      "title": "Provenance",
      "type": "string"
    },
    "predicted_classes": {
      "items": {
        "$ref": "#/$defs/PictureClassificationClass"
      },
      "title": "Predicted Classes",
      "type": "array"
    }
  },
  "required": [
    "provenance",
    "predicted_classes"
  ],
  "title": "PictureClassificationData",
  "type": "object"
},
"PictureClassificationMetaField": {

```



```

    "additionalProperties": true,
    "description": "Picture classification metadata field.",
    "properties": {
      "predictions": {
        "items": {
          "$ref": "#/$defs/PictureClassificationPrediction"
        },
        "minItems": 1,
        "title": "Predictions",
        "type": "array"
      }
    },
    "title": "PictureClassificationMetaField",
    "type": "object"
  },
  "PictureClassificationPrediction": {
    "additionalProperties": true,
    "description": "Picture classification instance.",
    "properties": {
      "confidence": {
        "anyOf": [
          {
            "maximum": 1,
            "minimum": 0,
            "type": "number"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The confidence of the prediction.",
        "examples": [
          0.9,
          0.42
        ],
        "title": "Confidence"
      },
      "created_by": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The origin of the prediction.",
        "examples": [
          "ibm-granite/granite-docling-258M"
        ],
        "title": "Created By"
      },
      "class_name": {
        "title": "Class Name",
        "type": "string"
      }
    },
    "required": [
      "class_name"
    ],
    "title": "PictureClassificationPrediction",
    "type": "object"
  },
  "PictureItem": {
    "additionalProperties": false,
    "description": "PictureItem.",
    "properties": {

```

```

"self_ref": {
  "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
  "title": "Self Ref",
  "type": "string"
},
"parent": {
  "anyOf": [
    {
      "$ref": "#/$defs/RefItem"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"children": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "Children",
  "type": "array"
},
"content_layer": {
  "$ref": "#/$defs/ContentLayer",
  "default": "body"
},
"meta": {
  "anyOf": [
    {
      "$ref": "#/$defs/PictureMeta"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"label": {
  "default": "picture",
  "enum": [
    "picture",
    "chart"
  ],
  "title": "Label",
  "type": "string"
},
"prov": {
  "default": [],
  "items": {
    "$ref": "#/$defs/ProvenanceItem"
  },
  "title": "Prov",
  "type": "array"
},
"source": {
  "default": [],
  "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
  "items": {
    "discriminator": {
      "mapping": {
        "track": "#/$defs/TrackSource"
      }
    },
    "propertyName": "kind"
  },
  "oneOf": [
    {

```

```

    "$ref": "#/$defs/TrackSource"
  },
  ],
  "title": "Source",
  "type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"captions": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "Captions",
  "type": "array"
},
"references": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "References",
  "type": "array"
},
"footnotes": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "Footnotes",
  "type": "array"
},
"image": {
  "anyOf": [
    {
      "$ref": "#/$defs/ImageRef"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"annotations": {
  "default": [],
  "deprecated": true,
  "items": {
    "discriminator": {
      "mapping": {
        "bar_chart_data": "#/$defs/PictureBarChartData",
        "classification": "#/$defs/PictureClassificationData",
        "description": "#/$defs/DescriptionAnnotation",
        "line_chart_data": "#/$defs/PictureLineChartData",
        "misc": "#/$defs/MiscAnnotation",
        "molecule_data": "#/$defs/PictureMoleculeData",
        "pie_chart_data": "#/$defs/PicturePieChartData",
        "scatter_chart_data": "#/$defs/PictureScatterChartData",
        "stacked_bar_chart_data": "#/$defs/PictureStackedBarChartData",
        "tabular_chart_data": "#/$defs/PictureTabularChartData"
      },
      "propertyName": "kind"
    }
  },

```

```

        "oneOf": [
            {
                "$ref": "#/$defs/DescriptionAnnotation"
            },
            {
                "$ref": "#/$defs/MiscAnnotation"
            },
            {
                "$ref": "#/$defs/PictureClassificationData"
            },
            {
                "$ref": "#/$defs/PictureMoleculeData"
            },
            {
                "$ref": "#/$defs/PictureTabularChartData"
            },
            {
                "$ref": "#/$defs/PictureLineChartData"
            },
            {
                "$ref": "#/$defs/PictureBarChartData"
            },
            {
                "$ref": "#/$defs/PictureStackedBarChartData"
            },
            {
                "$ref": "#/$defs/PicturePieChartData"
            },
            {
                "$ref": "#/$defs/PictureScatterChartData"
            }
        ],
        "title": "Annotations",
        "type": "array"
    },
    "required": [
        "self_ref"
    ],
    "title": "PictureItem",
    "type": "object"
},
"PictureLineChartData": {
    "description": "Represents data of a line chart.\n\nAttributes:\n    kind (Literal[\"line_chart_data\"]): The type of the chart.\n    x_axis_label (str): The label for the x-axis.\n    y_axis_label (str): The label for the y-axis.\n    lines (list[ChartLine]): A list of lines in the chart.",
    "properties": {
        "kind": {
            "const": "line_chart_data",
            "default": "line_chart_data",
            "title": "Kind",
            "type": "string"
        },
        "title": {
            "title": "Title",
            "type": "string"
        },
        "x_axis_label": {
            "title": "X Axis Label",
            "type": "string"
        },
        "y_axis_label": {
            "title": "Y Axis Label",
            "type": "string"
        },
        "lines": {
            "items": {
                "$ref": "#/$defs/ChartLine"
            }
        }
    }
}

```

```

    },
    "title": "Lines",
    "type": "array"
  },
},
"required": [
  "title",
  "x_axis_label",
  "y_axis_label",
  "lines"
],
"title": "PictureLineChartData",
"type": "object"
},
"PictureMeta": {
  "additionalProperties": true,
  "description": "Metadata model for pictures.",
  "properties": {
    "summary": {
      "anyOf": [
        {
          "$ref": "#/$defs/SummaryMetaField"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "description": {
      "anyOf": [
        {
          "$ref": "#/$defs/DescriptionMetaField"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "classification": {
      "anyOf": [
        {
          "$ref": "#/$defs/PictureClassificationMetaField"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "molecule": {
      "anyOf": [
        {
          "$ref": "#/$defs/MoleculeMetaField"
        },
        {
          "type": "null"
        }
      ],
      "default": null
    },
    "tabular_chart": {
      "anyOf": [
        {
          "$ref": "#/$defs/TabularChartMetaField"
        },
        {
          "type": "null"
        }
      ],

```

```

    },
    "default": null
  },
  },
  "title": "PictureMeta",
  "type": "object"
},
"PictureMoleculeData": {
  "description": "PictureMoleculeData.",
  "properties": {
    "kind": {
      "const": "molecule_data",
      "default": "molecule_data",
      "title": "Kind",
      "type": "string"
    },
    "smi": {
      "title": "Smi",
      "type": "string"
    },
    "confidence": {
      "title": "Confidence",
      "type": "number"
    },
    "class_name": {
      "title": "Class Name",
      "type": "string"
    },
    "segmentation": {
      "items": {
        "maxItems": 2,
        "minItems": 2,
        "prefixItems": [
          {
            "type": "number"
          },
          {
            "type": "number"
          }
        ],
        "type": "array"
      },
      "title": "Segmentation",
      "type": "array"
    },
    "provenance": {
      "title": "Provenance",
      "type": "string"
    }
  },
  "required": [
    "smi",
    "confidence",
    "class_name",
    "segmentation",
    "provenance"
  ],
  "title": "PictureMoleculeData",
  "type": "object"
},
"PicturePieChartData": {
  "description": "Represents data of a pie chart.\n\nAttributes:\n    kind (Literal[\"pie_chart_data\"]): The type of the chart.\n    slices (list[ChartSlice]): A list of slices in the pie chart.",
  "properties": {
    "kind": {
      "const": "pie_chart_data",
      "default": "pie_chart_data",
      "title": "Kind",

```

```

        "type": "string"
    },
    "title": {
        "title": "Title",
        "type": "string"
    },
    "slices": {
        "items": {
            "$ref": "#/$defs/ChartSlice"
        },
        "title": "Slices",
        "type": "array"
    }
},
"required": [
    "title",
    "slices"
],
"title": "PicturePieChartData",
"type": "object"
},
"PictureScatterChartData": {
    "description": "Represents data of a scatter chart.\n\nAttributes:\n    kind\n(Literal[\"scatter_chart_data\"]): The type of the chart.\n    x_axis_label (str): The label for the x-axis.\n    y_axis_label (str): The label for the y-axis.\n    points (list[ChartPoint]): A list of points in the scatter\nchart.",
    "properties": {
        "kind": {
            "const": "scatter_chart_data",
            "default": "scatter_chart_data",
            "title": "Kind",
            "type": "string"
        },
        "title": {
            "title": "Title",
            "type": "string"
        },
        "x_axis_label": {
            "title": "X Axis Label",
            "type": "string"
        },
        "y_axis_label": {
            "title": "Y Axis Label",
            "type": "string"
        },
        "points": {
            "items": {
                "$ref": "#/$defs/ChartPoint"
            },
            "title": "Points",
            "type": "array"
        }
    },
    "required": [
        "title",
        "x_axis_label",
        "y_axis_label",
        "points"
    ],
    "title": "PictureScatterChartData",
    "type": "object"
},
"PictureStackedBarChartData": {
    "description": "Represents data of a stacked bar chart.\n\nAttributes:\n    kind\n(Literal[\"stacked_bar_chart_data\"]): The type of the chart.\n    x_axis_label (str): The label for the x-\naxis.\n    y_axis_label (str): The label for the y-axis.\n    stacked_bars (list[ChartStackedBar]): A list of\nstacked bars in the chart.",
    "properties": {
        "kind": {

```

```

        "const": "stacked_bar_chart_data",
        "default": "stacked_bar_chart_data",
        "title": "Kind",
        "type": "string"
    },
    "title": {
        "title": "Title",
        "type": "string"
    },
    "x_axis_label": {
        "title": "X Axis Label",
        "type": "string"
    },
    "y_axis_label": {
        "title": "Y Axis Label",
        "type": "string"
    },
    "stacked_bars": {
        "items": {
            "$ref": "#/$defs/ChartStackedBar"
        },
        "title": "Stacked Bars",
        "type": "array"
    }
},
"required": [
    "title",
    "x_axis_label",
    "y_axis_label",
    "stacked_bars"
],
"title": "PictureStackedBarChartData",
"type": "object"
},
"PictureTabularChartData": {
    "description": "Base class for picture chart data.\n\nAttributes:\n    title (str): The title of the chart.\n    chart_data (TableData): Chart data in the table format.",
    "properties": {
        "kind": {
            "const": "tabular_chart_data",
            "default": "tabular_chart_data",
            "title": "Kind",
            "type": "string"
        },
        "title": {
            "title": "Title",
            "type": "string"
        },
        "chart_data": {
            "$ref": "#/$defs/TableData"
        }
    },
    "required": [
        "title",
        "chart_data"
    ],
    "title": "PictureTabularChartData",
    "type": "object"
},
"ProfilingItem": {
    "properties": {
        "scope": {
            "$ref": "#/$defs/ProfilingScope"
        },
        "count": {
            "default": 0,
            "title": "Count",
            "type": "integer"
        }
    },

```



```

    "times": {
      "default": [],
      "items": {
        "type": "number"
      },
      "title": "Times",
      "type": "array"
    },
    "start_timestamps": {
      "default": [],
      "items": {
        "format": "date-time",
        "type": "string"
      },
      "title": "Start Timestamps",
      "type": "array"
    }
  },
  "required": [
    "scope"
  ],
  "title": "ProfilingItem",
  "type": "object"
},
"ProfilingScope": {
  "enum": [
    "page",
    "document"
  ],
  "title": "ProfilingScope",
  "type": "string"
},
"ProvenanceItem": {
  "description": "Provenance information for elements extracted from a textual document.\n\nA
`ProvenanceItem` object acts as a lightweight pointer back into the original\ndocument for an extracted element.
It applies to documents with an explicit\nor implicit layout, such as PDF, HTML, docx, or pptx.",
  "properties": {
    "page_no": {
      "description": "Page number",
      "title": "Page No",
      "type": "integer"
    },
    "bbox": {
      "$ref": "#/$defs/BoundingBox",
      "description": "Bounding box"
    },
    "charspan": {
      "description": "Character span (0-indexed)",
      "maxItems": 2,
      "minItems": 2,
      "prefixItems": [
        {
          "type": "integer"
        },
        {
          "type": "integer"
        }
      ],
      "title": "Charspan",
      "type": "array"
    }
  },
  "required": [
    "page_no",
    "bbox",
    "charspan"
  ],
  "title": "ProvenanceItem",
  "type": "object"
}

```

```

    },
    "RefItem": {
      "description": "RefItem.",
      "properties": {
        "$ref": {
          "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
          "title": "$Ref",
          "type": "string"
        }
      },
      "required": [
        "$ref"
      ],
      "title": "RefItem",
      "type": "object"
    },
    "RichTableCell": {
      "description": "RichTableCell.",
      "properties": {
        "bbox": {
          "anyOf": [
            {
              "$ref": "#/$defs/BoundingBox"
            },
            {
              "type": "null"
            }
          ],
          "default": null
        },
        "row_span": {
          "default": 1,
          "title": "Row Span",
          "type": "integer"
        },
        "col_span": {
          "default": 1,
          "title": "Col Span",
          "type": "integer"
        },
        "start_row_offset_idx": {
          "title": "Start Row Offset Idx",
          "type": "integer"
        },
        "end_row_offset_idx": {
          "title": "End Row Offset Idx",
          "type": "integer"
        },
        "start_col_offset_idx": {
          "title": "Start Col Offset Idx",
          "type": "integer"
        },
        "end_col_offset_idx": {
          "title": "End Col Offset Idx",
          "type": "integer"
        },
        "text": {
          "title": "Text",
          "type": "string"
        },
        "column_header": {
          "default": false,
          "title": "Column Header",
          "type": "boolean"
        },
        "row_header": {
          "default": false,
          "title": "Row Header",
          "type": "boolean"
        }
      }
    }
  }
}

```

```

    },
    "row_section": {
      "default": false,
      "title": "Row Section",
      "type": "boolean"
    },
    "fillable": {
      "default": false,
      "title": "Fillable",
      "type": "boolean"
    },
    "ref": {
      "$ref": "#/$defs/RefItem"
    }
  },
  "required": [
    "start_row_offset_idx",
    "end_row_offset_idx",
    "start_col_offset_idx",
    "end_col_offset_idx",
    "text",
    "ref"
  ],
  "title": "RichTableCell",
  "type": "object"
},
"Script": {
  "description": "Text script position.",
  "enum": [
    "baseline",
    "sub",
    "super"
  ],
  "title": "Script",
  "type": "string"
},
"SectionHeaderItem": {
  "additionalProperties": false,
  "description": "SectionItem.",
  "properties": {
    "self_ref": {
      "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
      "title": "Self Ref",
      "type": "string"
    }
  },
  "parent": {
    "anyOf": [
      {
        "$ref": "#/$defs/RefItem"
      },
      {
        "type": "null"
      }
    ]
  },
  "default": null
},
"children": {
  "default": [],
  "items": {
    "$ref": "#/$defs/RefItem"
  },
  "title": "Children",
  "type": "array"
},
"content_layer": {
  "$ref": "#/$defs/ContentLayer",
  "default": "body"
},
"meta": {

```

```
    "anyOf": [
      {
        "$ref": "#/$defs/BaseMeta"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "label": {
    "const": "section_header",
    "default": "section_header",
    "title": "Label",
    "type": "string"
  },
  "prov": {
    "default": [],
    "items": {
      "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
  },
  "source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
    "items": {
      "discriminator": {
        "mapping": {
          "track": "#/$defs/TrackSource"
        }
      },
      "propertyName": "kind"
    },
    "oneOf": [
      {
        "$ref": "#/$defs/TrackSource"
      }
    ]
  },
  "title": "Source",
  "type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"orig": {
  "title": "Orig",
  "type": "string"
},
"text": {
  "title": "Text",
  "type": "string"
},
"formatting": {
  "anyOf": [
    {
      "$ref": "#/$defs/Formatting"
    },
    {
      "type": "null"
    }
  ]
},
```

```

    "default": null
  },
  "hyperlink": {
    "anyOf": [
      {
        "format": "uri",
        "minLength": 1,
        "type": "string"
      },
      {
        "format": "path",
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Hyperlink"
  },
  "level": {
    "default": 1,
    "maximum": 100,
    "minimum": 1,
    "title": "Level",
    "type": "integer"
  }
},
"required": [
  "self_ref",
  "orig",
  "text"
],
"title": "SectionHeaderItem",
"type": "object"
},
"SegmentedPdfPage": {
  "description": "Extended segmented page model specific to PDF documents.",
  "properties": {
    "dimension": {
      "$ref": "#/$defs/PdfPageGeometry"
    },
    "bitmap_resources": {
      "default": [],
      "items": {
        "$ref": "#/$defs/BitmapResource"
      },
      "title": "Bitmap Resources",
      "type": "array"
    },
    "char_cells": {
      "items": {
        "anyOf": [
          {
            "$ref": "#/$defs/PdfTextCell"
          },
          {
            "$ref": "#/$defs/TextCell"
          }
        ]
      },
      "title": "Char Cells",
      "type": "array"
    },
    "word_cells": {
      "items": {
        "anyOf": [
          {
            "$ref": "#/$defs/PdfTextCell"

```

```

    },
    {
      "$ref": "#/$defs/TextCell"
    }
  ],
  },
  "title": "Word Cells",
  "type": "array"
},
"textline_cells": {
  "items": {
    "anyOf": [
      {
        "$ref": "#/$defs/PdfTextCell"
      },
      {
        "$ref": "#/$defs/TextCell"
      }
    ]
  },
  "title": "Textline Cells",
  "type": "array"
},
"has_chars": {
  "default": false,
  "title": "Has Chars",
  "type": "boolean"
},
"has_words": {
  "default": false,
  "title": "Has Words",
  "type": "boolean"
},
"has_lines": {
  "default": false,
  "title": "Has Lines",
  "type": "boolean"
},
"image": {
  "anyOf": [
    {
      "$ref": "#/$defs/ImageRef"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"widgets": {
  "default": [],
  "items": {
    "$ref": "#/$defs/PdfWidget"
  },
  "title": "Widgets",
  "type": "array"
},
"hyperlinks": {
  "default": [],
  "items": {
    "$ref": "#/$defs/PdfHyperlink"
  },
  "title": "Hyperlinks",
  "type": "array"
},
"lines": {
  "default": [],
  "deprecated": true,
  "items": {

```

```

    "$ref": "#/$defs/PdfLine"
  },
  "title": "Lines",
  "type": "array"
},
"shapes": {
  "default": [],
  "items": {
    "$ref": "#/$defs/PdfShape"
  },
  "title": "Shapes",
  "type": "array"
}
},
"required": [
  "dimension",
  "char_cells",
  "word_cells",
  "textline_cells"
],
"title": "SegmentedPdfPage",
"type": "object"
},
"Size": {
  "description": "Size.",
  "properties": {
    "width": {
      "default": 0.0,
      "title": "Width",
      "type": "number"
    },
    "height": {
      "default": 0.0,
      "title": "Height",
      "type": "number"
    }
  },
  "title": "Size",
  "type": "object"
},
"SummaryMetaField": {
  "additionalProperties": true,
  "description": "Summary data.",
  "properties": {
    "confidence": {
      "anyOf": [
        {
          "maximum": 1,
          "minimum": 0,
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The confidence of the prediction.",
      "examples": [
        0.9,
        0.42
      ],
      "title": "Confidence"
    },
    "created_by": {
      "anyOf": [
        {
          "type": "string"
        },
        {

```

```

        "type": "null"
    },
    ],
    "default": null,
    "description": "The origin of the prediction.",
    "examples": [
        "ibm-granite/granite-docling-258M"
    ],
    "title": "Created By"
},
"text": {
    "title": "Text",
    "type": "string"
}
},
"required": [
    "text"
],
"title": "SummaryMetaField",
"type": "object"
},
"Table": {
    "properties": {
        "label": {
            "$ref": "#/$defs/DocItemLabel"
        },
        "id": {
            "title": "Id",
            "type": "integer"
        },
        "page_no": {
            "title": "Page No",
            "type": "integer"
        },
        "cluster": {
            "$ref": "#/$defs/Cluster"
        },
        "text": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Text"
        },
        "otsl_seq": {
            "items": {
                "type": "string"
            },
            "title": "Otsl Seq",
            "type": "array"
        },
        "num_rows": {
            "default": 0,
            "title": "Num Rows",
            "type": "integer"
        },
        "num_cols": {
            "default": 0,
            "title": "Num Cols",
            "type": "integer"
        },
        "table_cells": {
            "items": {
                "$ref": "#/$defs/TableCell"
            }
        }
    }
}

```



```

    },
    "title": "Table Cells",
    "type": "array"
  },
},
"required": [
  "label",
  "id",
  "page_no",
  "cluster",
  "otsl_seq",
  "table_cells"
],
"title": "Table",
"type": "object"
},
"TableCell": {
  "description": "TableCell.",
  "properties": {
    "bbox": {
      "anyOf": [
        {
          "$ref": "#/$defs/BoundingBox"
        },
        {
          "type": "null"
        }
      ]
    },
    "default": null
  },
},
"row_span": {
  "default": 1,
  "title": "Row Span",
  "type": "integer"
},
"col_span": {
  "default": 1,
  "title": "Col Span",
  "type": "integer"
},
"start_row_offset_idx": {
  "title": "Start Row Offset Idx",
  "type": "integer"
},
"end_row_offset_idx": {
  "title": "End Row Offset Idx",
  "type": "integer"
},
"start_col_offset_idx": {
  "title": "Start Col Offset Idx",
  "type": "integer"
},
"end_col_offset_idx": {
  "title": "End Col Offset Idx",
  "type": "integer"
},
"text": {
  "title": "Text",
  "type": "string"
},
"column_header": {
  "default": false,
  "title": "Column Header",
  "type": "boolean"
},
"row_header": {
  "default": false,
  "title": "Row Header",
  "type": "boolean"
}

```

```

    },
    "row_section": {
      "default": false,
      "title": "Row Section",
      "type": "boolean"
    },
    "fillable": {
      "default": false,
      "title": "Fillable",
      "type": "boolean"
    }
  },
  "required": [
    "start_row_offset_idx",
    "end_row_offset_idx",
    "start_col_offset_idx",
    "end_col_offset_idx",
    "text"
  ],
  "title": "TableCell",
  "type": "object"
},
"TableData": {
  "description": "BaseTableData.",
  "properties": {
    "table_cells": {
      "default": [],
      "items": {
        "anyOf": [
          {
            "$ref": "#/$defs/RichTableCell"
          },
          {
            "$ref": "#/$defs/TableCell"
          }
        ]
      }
    },
    "title": "Table Cells",
    "type": "array"
  },
  "num_rows": {
    "default": 0,
    "title": "Num Rows",
    "type": "integer"
  },
  "num_cols": {
    "default": 0,
    "title": "Num Cols",
    "type": "integer"
  }
},
"title": "TableData",
"type": "object"
},
"TableItem": {
  "additionalProperties": false,
  "description": "TableItem.",
  "properties": {
    "self_ref": {
      "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
      "title": "Self Ref",
      "type": "string"
    },
    "parent": {
      "anyOf": [
        {
          "$ref": "#/$defs/RefItem"
        },
        {

```

```

        "type": "null"
    },
    ],
    "default": null
},
"children": {
    "default": [],
    "items": {
        "$ref": "#/$defs/RefItem"
    },
    "title": "Children",
    "type": "array"
},
"content_layer": {
    "$ref": "#/$defs/ContentLayer",
    "default": "body"
},
"meta": {
    "anyOf": [
        {
            "$ref": "#/$defs/FloatingMeta"
        },
        {
            "type": "null"
        }
    ],
    "default": null
},
"label": {
    "default": "table",
    "enum": [
        "document_index",
        "table"
    ],
    "title": "Label",
    "type": "string"
},
"prov": {
    "default": [],
    "items": {
        "$ref": "#/$defs/ProvenanceItem"
    },
    "title": "Prov",
    "type": "array"
},
"source": {
    "default": [],
    "description": "The provenance of this document item. Currently, it is only used for media track
provenance.",
    "items": {
        "discriminator": {
            "mapping": {
                "track": "#/$defs/TrackSource"
            },
            "propertyName": "kind"
        },
        "oneOf": [
            {
                "$ref": "#/$defs/TrackSource"
            }
        ]
    },
    "title": "Source",
    "type": "array"
},
"comments": {
    "default": [],
    "items": {
        "$ref": "#/$defs/FineRef"
    }
}

```

```

    },
    "title": "Comments",
    "type": "array"
  },
  "captions": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Captions",
    "type": "array"
  },
  "references": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "References",
    "type": "array"
  },
  "footnotes": {
    "default": [],
    "items": {
      "$ref": "#/$defs/RefItem"
    },
    "title": "Footnotes",
    "type": "array"
  },
  "image": {
    "anyOf": [
      {
        "$ref": "#/$defs/ImageRef"
      },
      {
        "type": "null"
      }
    ],
    "default": null
  },
  "data": {
    "$ref": "#/$defs/TableData"
  },
  "annotations": {
    "default": [],
    "deprecated": true,
    "items": {
      "discriminator": {
        "mapping": {
          "description": "#/$defs/DescriptionAnnotation",
          "misc": "#/$defs/MiscAnnotation"
        },
        "propertyName": "kind"
      },
      "oneOf": [
        {
          "$ref": "#/$defs/DescriptionAnnotation"
        },
        {
          "$ref": "#/$defs/MiscAnnotation"
        }
      ]
    },
    "title": "Annotations",
    "type": "array"
  }
},
"required": [
  "self_ref",
  "data"
]

```

```

    },
    "title": "TableItem",
    "type": "object"
  },
  "TableStructurePrediction": {
    "properties": {
      "table_map": {
        "additionalProperties": {
          "$ref": "#/$defs/Table"
        },
        "default": {},
        "title": "Table Map",
        "type": "object"
      }
    },
    "title": "TableStructurePrediction",
    "type": "object"
  },
  "TabularChartMetaField": {
    "additionalProperties": true,
    "description": "Tabular chart metadata field.",
    "properties": {
      "confidence": {
        "anyOf": [
          {
            "maximum": 1,
            "minimum": 0,
            "type": "number"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The confidence of the prediction.",
        "examples": [
          0.9,
          0.42
        ],
        "title": "Confidence"
      },
      "created_by": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "description": "The origin of the prediction.",
        "examples": [
          "ibm-granite/granite-docling-258M"
        ],
        "title": "Created By"
      }
    },
    "title": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Title"
    }
  },

```

```

    "chart_data": {
      "$ref": "#/$defs/TableData"
    },
  },
  "required": [
    "chart_data"
  ],
  "title": "TabularChartMetaField",
  "type": "object"
},
"TextCell": {
  "description": "Model representing a text cell with positioning and content information.",
  "properties": {
    "index": {
      "default": -1,
      "title": "Index",
      "type": "integer"
    },
    "rgba": {
      "$ref": "#/$defs/ColorRGBA",
      "default": {
        "r": 0,
        "g": 0,
        "b": 0,
        "a": 255
      }
    },
    "rect": {
      "$ref": "#/$defs/BoundingBox"
    },
    "text": {
      "title": "Text",
      "type": "string"
    },
    "orig": {
      "title": "Orig",
      "type": "string"
    },
    "text_direction": {
      "$ref": "#/$defs/TextDirection",
      "default": "left_to_right"
    },
    "confidence": {
      "default": 1.0,
      "title": "Confidence",
      "type": "number"
    },
    "from_ocr": {
      "title": "From Ocr",
      "type": "boolean"
    }
  },
  "required": [
    "rect",
    "text",
    "orig",
    "from_ocr"
  ],
  "title": "TextCell",
  "type": "object"
},
"TextDirection": {
  "description": "Enumeration for text direction options.",
  "enum": [
    "left_to_right",
    "right_to_left",
    "unspecified"
  ],
  "title": "TextDirection",

```

```

    "type": "string"
  },
  "TextElement": {
    "properties": {
      "label": {
        "$ref": "#/$defs/DocItemLabel"
      },
      "id": {
        "title": "Id",
        "type": "integer"
      },
      "page_no": {
        "title": "Page No",
        "type": "integer"
      },
      "cluster": {
        "$ref": "#/$defs/Cluster"
      },
      "text": {
        "title": "Text",
        "type": "string"
      }
    },
    "required": [
      "label",
      "id",
      "page_no",
      "cluster",
      "text"
    ],
    "title": "TextElement",
    "type": "object"
  },
  "TextItem": {
    "additionalProperties": false,
    "description": "TextItem.",
    "properties": {
      "self_ref": {
        "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
        "title": "Self Ref",
        "type": "string"
      },
      "parent": {
        "anyOf": [
          {
            "$ref": "#/$defs/RefItem"
          },
          {
            "type": "null"
          }
        ],
        "default": null
      },
      "children": {
        "default": [],
        "items": {
          "$ref": "#/$defs/RefItem"
        },
        "title": "Children",
        "type": "array"
      },
      "content_layer": {
        "$ref": "#/$defs/ContentLayer",
        "default": "body"
      },
      "meta": {
        "anyOf": [
          {
            "$ref": "#/$defs/BaseMeta"

```

```

    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"label": {
  "enum": [
    "caption",
    "checkbox_selected",
    "checkbox_unselected",
    "footnote",
    "page_footer",
    "page_header",
    "paragraph",
    "reference",
    "text",
    "empty_value"
  ],
  "title": "Label",
  "type": "string"
},
"prov": {
  "default": [],
  "items": {
    "$ref": "#/$defs/ProvenanceItem"
  },
  "title": "Prov",
  "type": "array"
},
"source": {
  "default": [],
  "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
  "items": {
    "discriminator": {
      "mapping": {
        "track": "#/$defs/TrackSource"
      }
    },
    "propertyName": "kind"
  },
  "oneOf": [
    {
      "$ref": "#/$defs/TrackSource"
    }
  ]
},
"title": "Source",
"type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"orig": {
  "title": "Orig",
  "type": "string"
},
"text": {
  "title": "Text",
  "type": "string"
},
"formatting": {
  "anyOf": [

```



```

        {
            "$ref": "#/$defs/Formatting"
        },
        {
            "type": "null"
        }
    ],
    "default": null
},
"hyperlink": {
    "anyOf": [
        {
            "format": "uri",
            "minLength": 1,
            "type": "string"
        },
        {
            "format": "path",
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Hyperlink"
}
},
"required": [
    "self_ref",
    "label",
    "orig",
    "text"
],
"title": "TextItem",
"type": "object"
},
"TitleItem": {
    "additionalProperties": false,
    "description": "TitleItem.",
    "properties": {
        "self_ref": {
            "pattern": "^#(?:/([\\w-]+)(?:/(\\d+))?)?$",
            "title": "Self Ref",
            "type": "string"
        },
        "parent": {
            "anyOf": [
                {
                    "$ref": "#/$defs/RefItem"
                },
                {
                    "type": "null"
                }
            ],
            "default": null
        },
        "children": {
            "default": [],
            "items": {
                "$ref": "#/$defs/RefItem"
            },
            "title": "Children",
            "type": "array"
        },
        "content_layer": {
            "$ref": "#/$defs/ContentLayer",
            "default": "body"
        }
    },

```

```
"meta": {
  "anyOf": [
    {
      "$ref": "#/$defs/BaseMeta"
    },
    {
      "type": "null"
    }
  ],
  "default": null
},
"label": {
  "const": "title",
  "default": "title",
  "title": "Label",
  "type": "string"
},
"prov": {
  "default": [],
  "items": {
    "$ref": "#/$defs/ProvenanceItem"
  },
  "title": "Prov",
  "type": "array"
},
"source": {
  "default": [],
  "description": "The provenance of this document item. Currently, it is only used for media track provenance.",
  "items": {
    "discriminator": {
      "mapping": {
        "track": "#/$defs/TrackSource"
      }
    },
    "propertyName": "kind"
  },
  "oneOf": [
    {
      "$ref": "#/$defs/TrackSource"
    }
  ],
  "title": "Source",
  "type": "array"
},
"comments": {
  "default": [],
  "items": {
    "$ref": "#/$defs/FineRef"
  },
  "title": "Comments",
  "type": "array"
},
"orig": {
  "title": "Orig",
  "type": "string"
},
"text": {
  "title": "Text",
  "type": "string"
},
"formatting": {
  "anyOf": [
    {
      "$ref": "#/$defs/Formatting"
    },
    {
      "type": "null"
    }
  ]
}
```

```

    },
    "default": null
  },
  "hyperlink": {
    "anyOf": [
      {
        "format": "uri",
        "minLength": 1,
        "type": "string"
      },
      {
        "format": "path",
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Hyperlink"
  }
},
"required": [
  "self_ref",
  "orig",
  "text"
],
"title": "TitleItem",
"type": "object"
},
"TrackSource": {
  "description": "Source metadata for a cue extracted from a media track.\n\nA `TrackSource` instance identifies a cue in a media track (audio, video, subtitles, screen-recording captions,\netc.). A *cue* here refers to any discrete segment that was pulled out of the original asset, e.g., a subtitle\nblock, an audio clip, or a timed marker in a screen-recording.",
  "properties": {
    "kind": {
      "const": "track",
      "default": "track",
      "description": "Identifies this type of source.",
      "title": "Kind",
      "type": "string"
    },
    "start_time": {
      "description": "Start time offset of the track cue in seconds",
      "examples": [
        11.0,
        6.5,
        5370.0
      ],
      "title": "Start Time",
      "type": "number"
    },
    "end_time": {
      "description": "End time offset of the track cue in seconds",
      "examples": [
        12.0,
        8.2,
        5370.1
      ],
      "title": "End Time",
      "type": "number"
    },
    "identifier": {
      "anyOf": [
        {
          "type": "string"
        },
        {

```

```

        "type": "null"
    },
    ],
    "default": null,
    "description": "An identifier of the cue",
    "examples": [
        "test",
        "123",
        "b72d946"
    ],
    "title": "Identifier"
},
"voice": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "description": "The name of the voice in this track (the speaker)",
    "examples": [
        "John",
        "Mary",
        "Speaker 1"
    ],
    "title": "Voice"
}
),
"required": [
    "start_time",
    "end_time"
],
"title": "TrackSource",
"type": "object"
},
"VlmPrediction": {
    "properties": {
        "text": {
            "default": "",
            "title": "Text",
            "type": "string"
        },
        "generated_tokens": {
            "default": [],
            "items": {
                "$ref": "#/$defs/VlmPredictionToken"
            },
            "title": "Generated Tokens",
            "type": "array"
        },
        "generation_time": {
            "default": -1,
            "title": "Generation Time",
            "type": "number"
        },
        "num_tokens": {
            "anyOf": [
                {
                    "type": "integer"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Num Tokens"
        }
    }
}

```

```

    },
    "stop_reason": {
      "$ref": "#/$defs/VlmStopReason",
      "default": "unspecified"
    },
  },
  "input_prompt": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Input Prompt"
  }
},
"title": "VlmPrediction",
"type": "object"
},
"VlmPredictionToken": {
  "properties": {
    "text": {
      "default": "",
      "title": "Text",
      "type": "string"
    },
    "token": {
      "default": -1,
      "title": "Token",
      "type": "integer"
    },
    "logprob": {
      "default": -1,
      "title": "Logprob",
      "type": "number"
    }
  },
  "title": "VlmPredictionToken",
  "type": "object"
},
"VlmStopReason": {
  "enum": [
    "length",
    "stop_sequence",
    "end_of_sequence",
    "unspecified"
  ],
  "title": "VlmStopReason",
  "type": "string"
},
"XBRLBackendOptions": {
  "description": "Options specific to the XBRL backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    }
  },
  "kind": "const",
  "const": "xbrl",

```

```

        "default": "xbrl",
        "title": "Kind",
        "type": "string"
    },
    "taxonomy": {
        "anyOf": [
            {
                "format": "path",
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "description": "Path to a folder with the taxonomy required by the XBRL instance reports. It should include schemas (`.xsd`) and linkbases (`.xml`) referenced by the XBRL reports in their relative locations. Optionally, it can also include taxonomy packages (`.zip`) referenced by the reports with absolute URLs and mapped to files with a taxonomy catalog (`.catalog.xml`) for offline parsing.",
        "title": "Taxonomy"
    }
},
"title": "XBRLBackendOptions",
"type": "object"
}
},
"properties": {
    "version": {
        "$ref": "#/$defs/DoclingVersion",
        "default": {
            "docling_version": "2.80.0",
            "docling_core_version": "2.69.0",
            "docling_ibm_models_version": "3.12.0",
            "docling_parse_version": "5.5.0",
            "platform_str": "Linux-6.14.0-1017-azure-x86_64-with-glibc2.39",
            "py_impl_version": "cpython-312",
            "py_lang_version": "3.12.3"
        }
    },
    "timestamp": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Timestamp"
    },
    "status": {
        "$ref": "#/$defs/ConversionStatus",
        "default": "pending"
    },
    "errors": {
        "default": [],
        "items": {
            "$ref": "#/$defs/ErrorItem"
        },
        "title": "Errors",
        "type": "array"
    },
    "pages": {
        "default": [],
        "items": {
            "$ref": "#/$defs/Page"
        },
        "title": "Pages",

```

```

    "type": "array"
  },
  "timings": {
    "additionalProperties": {
      "$ref": "#/$defs/ProfilingItem"
    },
    "default": {},
    "title": "Timings",
    "type": "object"
  },
  "confidence": {
    "$ref": "#/$defs/ConfidenceReport"
  },
  "document": {
    "$ref": "#/$defs/DoclingDocument",
    "default": {
      "schema_name": "DoclingDocument",
      "version": "1.9.0",
      "name": "dummy",
      "origin": null,
      "furniture": {
        "children": [],
        "content_layer": "furniture",
        "label": "unspecified",
        "meta": null,
        "name": "_root_",
        "parent": null,
        "self_ref": "#/furniture"
      },
      "body": {
        "children": [],
        "content_layer": "body",
        "label": "unspecified",
        "meta": null,
        "name": "_root_",
        "parent": null,
        "self_ref": "#/body"
      },
      "groups": [],
      "texts": [],
      "pictures": [],
      "tables": [],
      "key_value_items": [],
      "form_items": [],
      "pages": {}
    }
  },
  "input": {
    "$ref": "#/$defs/InputDocument"
  },
  "assembled": {
    "$ref": "#/$defs/AssembledUnit",
    "default": {
      "elements": [],
      "body": [],
      "headers": []
    }
  }
},
"required": [
  "input"
],
"title": "ConversionResult",
"type": "object"
}

```

Fields:

- `version` (`DoclingVersion`)
- `timestamp` (`Optional[str]`)
- `status` (`ConversionStatus`)
- `errors` (`list[ErrorItem]`)
- `pages` (`list[Page]`)
- `timings` (`dict[str, ProfilingItem]`)
- `confidence` (`ConfidenceReport`)
- `document` (`DoclingDocument`)
- `input` (`InputDocument`)
- `assembled` (`AssembledUnit`)

attr assembled `pydantic-field`

`assembled: AssembledUnit`

attr confidence `pydantic-field`

`confidence: ConfidenceReport`

attr document `pydantic-field`

`document: DoclingDocument`

attr errors `pydantic-field`

`errors: list[ErrorItem]`

attr input `pydantic-field`

`input: InputDocument`

attr legacy_document `property`

`legacy_document`

attr pages `pydantic-field`

`pages: list[Page]`

attr status `pydantic-field`

`status: ConversionStatus`

attr timestamp `pydantic-field`

`timestamp: Optional[str]`

attr timings pydantic-field

```
timings: dict[str, ProfilingItem]
```

attr version pydantic-field

```
version: DoclingVersion
```

meth load classmethod

```
load(filename: Union[str, Path]) -> ConversionAssets
```

Load a ConversionAssets.

meth save

```
save(*, filename: Union[str, Path], indent: Optional[int] = 2)
```

Serialize the full ConversionAssets to JSON.

class ConversionStatus

Bases: str, Enum

Attributes:

- FAILURE -
- PARTIAL_SUCCESS -
- PENDING -
- SKIPPED -
- STARTED -
- SUCCESS -

attr FAILURE class-attribute instance-attribute

```
FAILURE
```

attr PARTIAL_SUCCESS class-attribute instance-attribute

```
PARTIAL_SUCCESS
```

attr PENDING class-attribute instance-attribute

```
PENDING
```

attr SKIPPED class-attribute instance-attribute

```
SKIPPED
```

attr **STARTED** `class-attribute` `instance-attribute`

STARTED



attr **SUCCESS** `class-attribute` `instance-attribute`

SUCCESS



class **FormatOption** `pydantic-model`

Bases: `BaseFormatOption`



```
{
  "$defs": {
    "AcceleratorDevice": {
      "description": "Devices to run model inference",
      "enum": [
        "auto",
        "cpu",
        "cuda",
        "mps",
        "xpu"
      ],
      "title": "AcceleratorDevice",
      "type": "string"
    },
    "AcceleratorOptions": {
      "additionalProperties": false,
      "description": "Hardware acceleration configuration for model inference.\n\nCan be configured via environment variables with DOCLING_ prefix.",
      "properties": {
        "num_threads": {
          "default": 4,
          "description": "Number of CPU threads to use for model inference. Higher values can improve throughput on multi-core systems but may increase memory usage. Can be set via DOCLING_NUM_THREADS or OMP_NUM_THREADS environment variables. Recommended: number of physical CPU cores.",
          "title": "Num Threads",
          "type": "integer"
        },
        "device": {
          "anyOf": [
            {
              "type": "string"
            },
            {
              "$ref": "#/$defs/AcceleratorDevice"
            }
          ],
          "default": "auto",
          "description": "Hardware device for model inference. Options: `auto` (automatic detection), `cpu` (CPU only), `cuda` (NVIDIA GPU), `cuda:N` (specific GPU), `mps` (Apple Silicon), `xpu` (Intel GPU). Auto mode selects the best available device. Can be set via DOCLING_DEVICE environment variable.",
          "title": "Device"
        },
        "cuda_use_flash_attention2": {
          "default": false,
          "description": "Enable Flash Attention 2 optimization for CUDA devices. Provides significant speedup and memory reduction for transformer models on compatible NVIDIA GPUs (Ampere or newer). Requires flash-attn package installation. Can be set via DOCLING_CUDA_USE_FLASH_ATTENTION2 environment variable.",
          "title": "Cuda Use Flash Attention2",
          "type": "boolean"
        }
      },
      "title": "AcceleratorOptions",
      "type": "object"
    },
    "DeclarativeBackendOptions": {
      "description": "Default backend options for a declarative document backend.",
      "properties": {
        "enable_remote_fetch": {
          "default": false,
          "description": "Enable remote resource fetching.",
          "title": "Enable Remote Fetch",
          "type": "boolean"
        },
        "enable_local_fetch": {
          "default": false,
```

```
    "description": "Enable local resource fetching.",
    "title": "Enable Local Fetch",
    "type": "boolean"
  },
  "kind": {
    "const": "declarative",
    "default": "declarative",
    "title": "Kind",
    "type": "string"
  }
},
"title": "DeclarativeBackendOptions",
"type": "object"
},
"HTMLBackendOptions": {
  "description": "Options specific to the HTML backend.\n\nThis class can be extended to include options specific to HTML processing.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "html",
      "default": "html",
      "title": "Kind",
      "type": "string"
    },
    "fetch_images": {
      "default": false,
      "description": "Whether the backend should access remote or local resources to parse images in an HTML document.",
      "title": "Fetch Images",
      "type": "boolean"
    },
    "source_uri": {
      "anyOf": [
        {
          "format": "uri",
          "minLength": 1,
          "type": "string"
        },
        {
          "format": "path",
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The URI that originates the HTML document. If provided, the backend will use it to resolve relative paths in the HTML document.",
      "title": "Source Uri"
    },
    "add_title": {
      "default": true,
      "description": "Add the HTML title tag as furniture in the DoclingDocument.",
      "title": "Add Title",
      "type": "boolean"
    }
  }
},
```

```

    "infer_furniture": {
      "default": true,
      "description": "Infer all the content before the first header as furniture.",
      "title": "Infer Furniture",
      "type": "boolean"
    },
  },
  "title": "HTMLBackendOptions",
  "type": "object"
},
"LatexBackendOptions": {
  "description": "Options specific to the LaTeX backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "latex",
      "default": "latex",
      "title": "Kind",
      "type": "string"
    },
    "parse_timeout": {
      "anyOf": [
        {
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": 30.0,
      "description": "Maximum time allowed for parsing a LaTeX document. Set to None to disable the timeout. Defaults to 30 s.",
      "title": "Parse Timeout"
    }
  },
  "title": "LatexBackendOptions",
  "type": "object"
},
"MarkdownBackendOptions": {
  "description": "Options specific to the Markdown backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "md",
      "default": "md",
      "title": "Kind",
      "type": "string"
    }
  }
}

```

```

    },
    "fetch_images": {
      "default": false,
      "description": "Whether the backend should access remote or local resources to parse images in the
markdown document.",
      "title": "Fetch Images",
      "type": "boolean"
    },
    "source_uri": {
      "anyOf": [
        {
          "format": "uri",
          "minLength": 1,
          "type": "string"
        },
        {
          "format": "path",
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "The URI that originates the markdown document. If provided, the backend will use it to
resolve relative paths in the markdown document.",
      "title": "Source Uri"
    }
  },
  "title": "MarkdownBackendOptions",
  "type": "object"
},
"MsExcelBackendOptions": {
  "description": "Options specific to the MS Excel backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "xlsx",
      "default": "xlsx",
      "title": "Kind",
      "type": "string"
    },
    "treat_singleton_as_text": {
      "default": false,
      "description": "Whether to treat singleton cells (1x1 tables with empty neighboring cells) as TextItem
instead of TableItem.",
      "title": "Treat Singleton As Text",
      "type": "boolean"
    },
    "gap_tolerance": {
      "default": 0,
      "description": "The tolerance (in number of empty rows/columns) for merging nearby data clusters into
a single table. Default is 0 (strict).",
      "title": "Gap Tolerance",
      "type": "integer"
    }
  },
  "title": "MsExcelBackendOptions",

```

```

"type": "object"
},
"PdfBackendOptions": {
  "description": "Backend options for pdf document backends.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",
      "title": "Enable Local Fetch",
      "type": "boolean"
    },
    "kind": {
      "const": "pdf",
      "default": "pdf",
      "title": "Kind",
      "type": "string"
    },
    "password": {
      "anyOf": [
        {
          "format": "password",
          "type": "string",
          "writeOnly": true
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Password"
    }
  },
  "title": "PdfBackendOptions",
  "type": "object"
},
"PipelineOptions": {
  "description": "Base configuration for document processing pipelines.\n\nProvides the foundational settings shared by every pipeline type:\ndocument-level timeout, hardware accelerator selection, remote service\npermissions, external plugin control, and model artifact paths. All\nspecialized pipeline option classes inherit from this base.\n\nSee Also:\n  `ConvertPipelineOptions`: Adds picture classification and description.\n  `AsrPipelineOptions`: Audio/speech recognition pipeline.\n  `VlmExtractionPipelineOptions`: VLM-based structured extraction.",
  "properties": {
    "document_timeout": {
      "anyOf": [
        {
          "type": "number"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "Maximum processing time in seconds before aborting document conversion. When exceeded, the pipeline stops processing and returns partial results with PARTIAL_SUCCESS status. If None, no timeout is enforced. Recommended: 90-120 seconds for production systems.",
      "examples": [
        10.0,
        20.0
      ],
      "title": "Document Timeout"
    },
    "accelerator_options": {

```

```

    "$ref": "#/$defs/AcceleratorOptions",
    "default": {
      "num_threads": 4,
      "device": "auto",
      "cuda_use_flash_attention2": false
    },
    "description": "Hardware acceleration configuration for model inference. Controls GPU device selection, memory management, and execution optimization settings for layout, OCR, and table structure models."
  },
  "enable_remote_services": {
    "default": false,
    "description": "Allow pipeline to call external APIs or cloud services during processing. Required for API-based picture description models. Disabled by default for security and offline operation.",
    "examples": [
      false
    ],
    "title": "Enable Remote Services",
    "type": "boolean"
  },
  "allow_external_plugins": {
    "default": false,
    "description": "Allow loading external third-party plugins for OCR, layout, table structure, or picture description models. Enables custom model implementations via plugin system. Disabled by default for security.",
    "examples": [
      false
    ],
    "title": "Allow External Plugins",
    "type": "boolean"
  },
  "artifacts_path": {
    "anyOf": [
      {
        "format": "path",
        "type": "string"
      },
      {
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "description": "Local directory containing pre-downloaded model artifacts (weights, configs). If None, models are fetched from remote sources on first use. Use `docling-tools models download` to pre-fetch artifacts for offline operation or faster initialization.",
    "examples": [
      "./artifacts",
      "/tmp/docling_outputs"
    ],
    "title": "Artifacts Path"
  }
},
"title": "PipelineOptions",
"type": "object"
},
"XBRLBackendOptions": {
  "description": "Options specific to the XBRL backend.",
  "properties": {
    "enable_remote_fetch": {
      "default": false,
      "description": "Enable remote resource fetching.",
      "title": "Enable Remote Fetch",
      "type": "boolean"
    },
    "enable_local_fetch": {
      "default": false,
      "description": "Enable local resource fetching.",

```



```

        "title": "Enable Local Fetch",
        "type": "boolean"
    },
    "kind": {
        "const": "xbrl",
        "default": "xbrl",
        "title": "Kind",
        "type": "string"
    },
    "taxonomy": {
        "anyOf": [
            {
                "format": "path",
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "description": "Path to a folder with the taxonomy required by the XBRL instance reports. It should include schemas (`.xsd`) and linkbases (`.xml`) referenced by the XBRL reports in their relative locations. Optionally, it can also include taxonomy packages (`.zip`) referenced by the reports with absolute URLs and mapped to files with a taxonomy catalog (`.catalog.xml`) for offline parsing.",
        "title": "Taxonomy"
    }
},
"title": "XBRLBackendOptions",
"type": "object"
}
},
"properties": {
    "pipeline_options": {
        "anyOf": [
            {
                "$ref": "#/$defs/PipelineOptions"
            },
            {
                "type": "null"
            }
        ],
        "default": null
    },
    "backend": {
        "title": "Backend"
    },
    "pipeline_cls": {
        "title": "Pipeline Cls"
    },
    "backend_options": {
        "anyOf": [
            {
                "discriminator": {
                    "mapping": {
                        "declarative": "#/$defs/DeclarativeBackendOptions",
                        "html": "#/$defs/HTMLBackendOptions",
                        "latex": "#/$defs/LatexBackendOptions",
                        "md": "#/$defs/MarkdownBackendOptions",
                        "pdf": "#/$defs/PdfBackendOptions",
                        "xbrl": "#/$defs/XBRLBackendOptions",
                        "xlsx": "#/$defs/MsExcelBackendOptions"
                    }
                },
                "propertyName": "kind"
            },
            {
                "$ref": "#/$defs/DeclarativeBackendOptions"
            }
        ]
    }
}

```

```

        "$ref": "#/$defs/HTMLBackendOptions"
    },
    {
        "$ref": "#/$defs/MarkdownBackendOptions"
    },
    {
        "$ref": "#/$defs/PdfBackendOptions"
    },
    {
        "$ref": "#/$defs/MsExcelBackendOptions"
    },
    {
        "$ref": "#/$defs/LatexBackendOptions"
    },
    {
        "$ref": "#/$defs/XBRLBackendOptions"
    }
]
},
{
    "type": "null"
}
],
"default": null,
"title": "Backend Options"
}
),
"required": [
    "backend",
    "pipeline_cls"
],
"title": "FormatOption",
"type": "object"
}
}

```

Fields:

- `pipeline_options` (Optional[PipelineOptions])
- `backend` (Type[AbstractDocumentBackend])
- `pipeline_cls` (Type[BasePipeline])
- `backend_options` (Optional[BackendOptions])

Validators:

- `set_optional_field_default`

attr backend pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr backend_options pydantic-field

```
backend_options: Optional[BackendOptions]
```

attr model_config class-attribute instance-attribute

```
model_config
```

attr pipeline_cls pydantic-field

```
pipeline_cls: Type[BasePipeline]
```

attr **pipeline_options** pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth **set_optional_field_default** pydantic-validator

```
set_optional_field_default() -> Self
```

class InputFormat

Bases: `str`, `Enum`

A document format supported by document backend parsers.

Attributes:

- `ASCIIDOC` -
- `AUDIO` -
- `CSV` -
- `DOCX` -
- `HTML` -
- `IMAGE` -
- `JSON_DOCLING` -
- `LATEX` -
- `MD` -
- `METS_GBS` -
- `PDF` -
- `PPTX` -
- `VTT` -
- `XLSX` -
- `XML_JATS` -
- `XML_USPTO` -
- `XML_XBRL` -

attr **ASCIIDOC** class-attribute instance-attribute

```
ASCIIDOC
```

attr **AUDIO** class-attribute instance-attribute

```
AUDIO
```

attr **CSV** class-attribute instance-attribute

CSV



attr DOCX class-attribute instance-attribute

DOCX



attr HTML class-attribute instance-attribute

HTML



attr IMAGE class-attribute instance-attribute

IMAGE



attr JSON_DOCLING class-attribute instance-attribute

JSON_DOCLING



attr LATEX class-attribute instance-attribute

LATEX



attr MD class-attribute instance-attribute

MD



attr METS_GBS class-attribute instance-attribute

METS_GBS



attr PDF class-attribute instance-attribute

PDF



attr PPTX class-attribute instance-attribute

PPTX



attr VTT class-attribute instance-attribute

VTT



attr XLSX class-attribute instance-attribute

XLSX



attr XML_JATS class-attribute instance-attribute

XML_JATS



attr XML_USPTO class-attribute instance-attribute

XML_USPTO

attr XML_XBRL class-attribute instance-attribute

XML_XBRL

class PdfFormatOption pydantic-model

Bases: [FormatOption](#)

Fields:

- `pipeline_options` (Optional[[PipelineOptions](#)])
- `pipeline_cls` (Type)
- `backend` (Type[[AbstractDocumentBackend](#)])
- `backend_options` (Optional[[PdfBackendOptions](#)])

Validators:

- `set_optional_field_default`

attr backend pydantic-field

backend: Type[[AbstractDocumentBackend](#)]

attr backend_options pydantic-field

backend_options: Optional[[PdfBackendOptions](#)]

attr model_config class-attribute instance-attribute

model_config

attr pipeline_cls pydantic-field

pipeline_cls: Type

attr pipeline_options pydantic-field

pipeline_options: Optional[[PipelineOptions](#)]

meth set_optional_field_default pydantic-validator

set_optional_field_default() -> Self

class ImageFormatOption pydantic-model

Bases: [FormatOption](#)

Fields:

- `pipeline_options` (Optional[PipelineOptions])
- `backend_options` (Optional[BackendOptions])
- `pipeline_cls` (Type)
- `backend` (Type[AbstractDocumentBackend])

Validators:

- `set_optional_field_default`

attr `backend` pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr `backend_options` pydantic-field

```
backend_options: Optional[BackendOptions]
```

attr `model_config` class-attribute instance-attribute

```
model_config
```

attr `pipeline_cls` pydantic-field

```
pipeline_cls: Type
```

attr `pipeline_options` pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth `set_optional_field_default` pydantic-validator

```
set_optional_field_default() -> Self
```

class `StandardPdfPipeline`

```
StandardPdfPipeline(pipeline_options: ThreadedPdfPipelineOptions)
```

Bases: `ConvertPipeline`

High-performance PDF pipeline with multi-threaded stages.

Methods:

- `execute` -
- `get_default_options` -
- `is_backend_supported` -

Attributes:

- `artifacts_path` (`Optional[Path]`) –
- `build_pipe` (`List[Callable]`) –
- `enrichment_pipe` –
- `keep_images` –
- `pipeline_options` (`ThreadedPdfPipelineOptions`) –

attr `artifacts_path` instance-attribute

`artifacts_path: Optional[Path]`

attr `build_pipe` instance-attribute

`build_pipe: List[Callable]`

attr `enrichment_pipe` instance-attribute

`enrichment_pipe`

attr `keep_images` instance-attribute

`keep_images`

attr `pipeline_options` instance-attribute

`pipeline_options: ThreadedPdfPipelineOptions`

meth `execute`

`execute(in_doc: InputDocument, raises_on_error: bool) -> ConversionResult`

meth `get_default_options` classmethod

`get_default_options() -> ThreadedPdfPipelineOptions`

meth `is_backend_supported` classmethod

`is_backend_supported(backend: AbstractDocumentBackend) -> bool`

class `WordFormatOption` pydantic-model

Bases: `FormatOption`

Fields:

- `pipeline_options` (`Optional[PipelineOptions]`)
- `backend_options` (`Optional[BackendOptions]`)
- `pipeline_cls` (`Type`)
- `backend` (`Type[AbstractDocumentBackend]`)

Validators:

- `set_optional_field_default`

attr backend pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr backend_options pydantic-field

```
backend_options: Optional[BackendOptions]
```

attr model_config class-attribute instance-attribute

```
model_config
```

attr pipeline_cls pydantic-field

```
pipeline_cls: Type
```

attr pipeline_options pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth set_optional_field_default pydantic-validator

```
set_optional_field_default() -> Self
```

class PowerpointFormatOption pydantic-model

Bases: `FormatOption`

Fields:

- `pipeline_options` (`Optional[PipelineOptions]`)
- `backend_options` (`Optional[BackendOptions]`)
- `pipeline_cls` (`Type`)
- `backend` (`Type[AbstractDocumentBackend]`)

Validators:

- `set_optional_field_default`

attr backend pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr backend_options pydantic-field

```
backend_options: Optional[BackendOptions]
```


attr model_config class-attribute instance-attribute

```
model_config
```

attr pipeline_cls pydantic-field

```
pipeline_cls: Type
```

attr pipeline_options pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth set_optional_field_default pydantic-validator

```
set_optional_field_default() -> Self
```

class MarkdownFormatOption pydantic-model

Bases: `FormatOption`

Fields:

- `pipeline_options` (Optional[PipelineOptions])
- `pipeline_cls` (Type)
- `backend` (Type[AbstractDocumentBackend])
- `backend_options` (Optional[MarkdownBackendOptions])

Validators:

- `set_optional_field_default`

attr backend pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr backend_options pydantic-field

```
backend_options: Optional[MarkdownBackendOptions]
```

attr model_config class-attribute instance-attribute

```
model_config
```

attr pipeline_cls pydantic-field

```
pipeline_cls: Type
```

attr pipeline_options pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth **set_optional_field_default** pydantic-validator

```
set_optional_field_default() -> Self
```

class **AsciiDocFormatOption** pydantic-model

Bases: [FormatOption](#)

Fields:

- **pipeline_options** (Optional[[PipelineOptions](#)])
- **backend_options** (Optional[[BackendOptions](#)])
- **pipeline_cls** (Type)
- **backend** (Type[[AbstractDocumentBackend](#)])

Validators:

- **set_optional_field_default**

attr **backend** pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr **backend_options** pydantic-field

```
backend_options: Optional[BackendOptions]
```

attr **model_config** class-attribute instance-attribute

```
model_config
```

attr **pipeline_cls** pydantic-field

```
pipeline_cls: Type
```

attr **pipeline_options** pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth **set_optional_field_default** pydantic-validator

```
set_optional_field_default() -> Self
```

class **HTMLFormatOption** pydantic-model

Bases: [FormatOption](#)

Fields:

- **pipeline_options** (Optional[[PipelineOptions](#)])

- `pipeline_cls` (`Type`)
- `backend` (`Type[AbstractDocumentBackend]`)
- `backend_options` (`Optional[HTMLBackendOptions]`)

Validators:

- `set_optional_field_default`

attr `backend` pydantic-field

```
backend: Type[AbstractDocumentBackend]
```

attr `backend_options` pydantic-field

```
backend_options: Optional[HTMLBackendOptions]
```

attr `model_config` class-attribute instance-attribute

```
model_config
```

attr `pipeline_cls` pydantic-field

```
pipeline_cls: Type
```

attr `pipeline_options` pydantic-field

```
pipeline_options: Optional[PipelineOptions]
```

meth `set_optional_field_default` pydantic-validator

```
set_optional_field_default() -> Self
```

class `SimplePipeline`

```
SimplePipeline(pipeline_options: ConvertPipelineOptions)
```

Bases: `ConvertPipeline`

`SimpleModelPipeline`.

This class is used at the moment for formats / backends which produce straight DoclingDocument output.

Methods:

- `execute` -
- `get_default_options` -
- `is_backend_supported` -

Attributes:

- `artifacts_path` (`Optional[Path]`) -

- `build_pipe` (`List[Callable]`) –
- `enrichment_pipe` –
- `keep_images` –
- `pipeline_options` (`ConvertPipelineOptions`) –

attr `artifacts_path` instance-attribute

```
artifacts_path: Optional[Path]
```

attr `build_pipe` instance-attribute

```
build_pipe: List[Callable]
```

attr `enrichment_pipe` instance-attribute

```
enrichment_pipe
```

attr `keep_images` instance-attribute

```
keep_images
```

attr `pipeline_options` instance-attribute

```
pipeline_options: ConvertPipelineOptions
```

meth `execute`

```
execute(in_doc: InputDocument, raises_on_error: bool) -> ConversionResult
```

meth `get_default_options` classmethod

```
get_default_options() -> ConvertPipelineOptions
```

meth `is_backend_supported` classmethod

```
is_backend_supported(backend: AbstractDocumentBackend)
```